



Phison Electronics Corporation aiDAPTIVLink2.0 NXUN205.A1 Install SOP & User manual

Version 1.0

Phison Electronics Corporation

Tel: +886-37-586-896 Fax: +886-37-587-868

E-mail: sales@phison.com / support@phison.com

Phison may make changes to specifications and product description at any time without notice. PHISON and the Phison logo are trademarks of Phison Electronics Corporation, registered in the United States and other countries. Products and specifications discussed herein are for reference purposes only. Copies of documents which include information of part number or ordering number, or other materials may be obtained by emailing us at sales@phison.com or support@phison.com.

©2024 Phison Electronics Corp. All Rights Reserved.

PHISON Confidential

REVISION HISTORY

Revision	Draft Date	History	aiDAPTIVLink Version	Author
1.0	2026/03/13	First Release	NXUN205.A1	Heine Chu

PHISON Confidential

TABLE OF CONTENTS

REVISION HISTORY	3
TABLE OF CONTENTS	4
1. CHECK DEVICE	8
1.1. Check GPU is in Approved Vendor List? (AVL)	8
1.2. Check the number of GPUs	8
1.3. Check DRAM and aiDAPTIVCache from different LLM model size	8
1.4. Check CPU configuration	11
2. INSTALLATION AND PREPARATION	12
2.1. Environment Preparation	12
2.1.1. Requirements	12
2.1.2. Install GPU Driver	12
2.1.3. Install GPU Toolkit : CUDA	13
2.1.4. Install GPU Toolkit : cuDNN (9.17)	14
2.2. Disk Setup (LVM Setting)	14
2.3. Login Huggingface	17
2.3.1. How to get Hugging Face token	17
2.3.2. How to register the Hugging Face token	20
2.4. Download Llama-3.1-8B-Instruct	21
2.4.1. Download Llama-3.1-8B-Instruct model	21
2.5. aiDAPTIV Middleware Installation	23
2.5.1. Local Depoly aiDAPTIV	23
2.5.2. Train model in Docker	25
2.6. Run aiDAPTIV	29
2.7. Quick Start for pytorch	41
2.7.1. Without aiDAPTIV+ Middleware	41
2.7.2. With aiDAPTIV+ Middleware	41
3. BENCHMARK BY AIDAPTIV TOOLKIT (OPTIONAL)	42
3.1. aiDAPTIV Toolkit Installation flow	42
3.1.1. aiDAPTIV Toolkit	42
3.1.2. Unzip download file	42

3.1.3.	Change to folder	42
3.1.4.	Deploy aiDAPTIV Toolkit and related library	42
3.2.	Start aiDAPTIV Toolkit	43
3.2.1.	Modify training setting in toolkit_v2.3.1_python312/project.ini	43
3.2.2.	Enter aiDAPTIV Toolkit folder	43
3.2.3.	Run aiDAPTIV Toolkit Performance Test.....	44
3.3.	Performance Result	45
3.3.1.	Log directory Description	45
3.3.2.	Performance.....	46
3.3.3.	Reference Data.....	47
4.	QA DATASETS	48
4.1.	QA datasets	48
4.2.	Pretrain datasets.....	49
4.3.	RAG datasets	50
4.4.	Harmony datasets.....	52
4.5.	example of datasets.....	54
4.5.1.	Example 1 (QA dataset)	54
4.5.2.	Example 2 (RAG dataset)	54
4.5.3.	Example 3 (nested RAG dataset)	55
4.5.4.	example 4 (Harmony dataset)	56
4.6.	Compare Change	57
4.6.1.	Previous dataloader on Dahoas/rm-static after apply template	57
4.6.2.	Current dataloader on Dahoas/rm-static after apply template	58
4.6.3.	Harmony dataset (HuggingFaceH4/Multilingual-Thinking) before applying template.	59
4.6.4.	Harmony dataset (HuggingFaceH4/Multilingual-Thinking) after applying Harmony template	60
5.	MULTIMODALITY DATASET CONFIG.....	61
5.1.	VQA datasets	61
5.2.	Description for each value.....	62
5.3.	Supported dataset	62
5.3.1.	Custom dataset: The data path should be in the format of ./json or ./csv** to properly parse	

<i>the data</i>	63
5.3.2. <i>Example (MathVista QA dataset)</i>	63
5.3.3. <i>dataset example</i>	64
5.3.4. <i>dataset structure example</i>	65
5.3.5. <i>Custom dataset config example</i>	65
APPENDIX A. HOW TO SET SWAP FILE	66
APPENDIX B. APPROVED VENDOR LIST (AVL)	67
B.1 GPU AVL	67
B.2 CPU AVL	67
B.3 Support Model list	68
APPENDIX C. FW UPDATE	69
Q&A EN	71

Preface

Purpose

This manual is intended solely for aiDAPTIV partners.

- Middleware Version: vNXUN_2_05.A1
- aiDAPTIV Toolkit Version: v2.3.1
- aiDAPTIVCache Family :

Model
AI100E SSD
AI200E SSD

1. CHECK DEVICE

In this section, we will check machine device GPU/CPU/RAM/aiDAPTIVCache before start to use aiDAPTIV middleware.

After completing this section, you will know which GPU/CPU is in Phison AVL, and how many RAM size for different LLM model.

1.1. Check GPU is in Approved Vendor List? (AVL)

- Check GPU is in AVL list ([Appendix B - GPU AVL](#)).
- Needs to do validation if GPU is not in AVL. Please refer Appendix E.
- It is recommended that the GPU must be connected to the PCIe Slot of X16. Connecting other Slots (ex: X8, X4) will decrease GPU performance.

1.2. Check the number of GPUs

- Number of GPUs = 2^n ($n=0,1,2,3,4$, GPUs = 1,2,4,8,16)

1.3. Check DRAM and aiDAPTIVCache from different LLM model size

- DRAM and aiDAPTIVCache from different LLM model size

LLM model size	<13B	34B	70B	180B
DRAM	64GB	64GB	128GB	128GB
aiDAPTIVCache capacity	1TB	1TB	2TB	2TB
AiDAPTIVCache count	1	2	2	4
aiDAPTIVCache slot count	1	2	2	4

- Workstation or PC : Use m.2 slot or m.2 PCIe expansion card.
 - <https://www.asus.com/tw/motherboards-components/motherboards/accessories/hyper-m-2-x16-gen5-card/>
- Rack server: Use U.2 slot.
- Recommend Gen4 or above.
- Recommend DRAM 2933MHz or above.
- Recommended DRAM size is 128GB or more
- Recommended DRAM channel number is 8 or more, ex: 16GB x8
- Logic for aiDAPTIVCache sizing:
 - The storage required for aiDAPTIVCache is primarily dictated by the model's weights, gradients, and optimizer states, which collectively form a core baseline of approximately 16x to 20x the model size.

- Under a Mixed-Precision training regime, this footprint is composed of:
 - Parameters & Gradients:
 - Maintained in both BF16 (p_bf16,g_bf16) for computational efficiency and FP32 (p_fp32) as master weights to ensure numerical stability.
 - Optimizer States (Adam):
 - Includes Momentum (m_fp32) and Variance (v_fp32) stored in FP32.
 - Runtime Activations:
 - Dynamic memory that scales with the batch sizes.
- While the (20 times) multiplier serves as a reliable estimation benchmark for large-scale model training, the total occupancy fluctuates based on the activations. Under high-load configurations (e.g., maximum batch sizes), the total usage may exceed this baseline due to the substantial growth of activation data.
- Empirical Validation:
 - The following table illustrates the relationship between model parameters and actual cache usage using 4x RTX 4000 Ada as an example. In this distributed setup, the specific components (p,g,m,v) are partitioned and calculated per GPU (per rank).
 - The data confirms that while core states remain a fixed baseline, total cache requirements scale dynamically with the chosen batch sizes.

LLM model	batch size	GPU num	p_bf16	p_fp32	g_bf16	m_fp32	v_fp32	activation	Total aiDAPTIVCache used
Meta-Llama-3.1-8B-Instruct	2	4	3.9GB	7.8GB	3.9GB	7.8GB	7.8GB	1.1GB	129.2GB, which is closed to $16 \times 8B$ ($\approx 128GB$) estimation.
Meta-Llama-3.1-8B-Instruct	56(max)	4	3.9GB	7.8GB	3.9GB	7.8GB	7.8GB	29GB	240.8GB, which exceeds $20 \times 8B$ ($\approx 160GB$) estimation.
Meta-Llama-3.1-70B-Instruct	2	4	33GB	66GB	33GB	66GB	66GB	5GB	1.076TB, which is closed to $16 \times 70B$ ($\approx 1.1TB$) estimation.
Meta-Llama-	23(max)	4	33GB	66GB	33GB	66GB	66GB	59GB	1.29TB, which is closed to $20 \times$

3.1-70B-Instruct									70B(≈1.4TB) estimation.
------------------	--	--	--	--	--	--	--	--	-------------------------

PHISON Confidential

1.4. Check CPU configuration

- Check CPU is in AVL list ([Appendix D - CPU AVL](#)).
- Recommend CPU Cores minimum 8 or above

CPU lanes calculate by GPU count and aiDAPTIVCache count.

Total PCIe lanes = GPU count * 16 + aiDAPTIVCache count * 4

*Example : For GPU count =4 and 70B LLM model size (aiDAPTIVCache 2)

Total $4 * 16 + 2 * 4 = 72$ lanes, It means at least 72 lanes for optimal performance.

PHISON Confidential

2. INSTALLATION AND PREPARATION

In this section, we will start installing the aiDAPTIV middleware and explain how to use the middleware. After completing this section, you will know how to use aiDAPTIV for Domain training or finetuning.

2.1. Environment Preparation

2.1.1. Requirements

- Recommend environment

Category	Detail
OS	Ubuntu 24.04.3 Desktop (kernel version: 6.14.0-28-generic)
NVIDIA GPU driver	NVIDIA driver version 580 installation
Python	3.12

2.1.2. Install GPU Driver

For NVIDIA series GPU

- Install Driver (Estimated time: 5 min)
 - Install NVIDIA Driver

```
sudo apt install nvidia-utils-580
sudo apt install nvidia-driver-580-open
```

- Reboot system

```
sudo reboot
```

- Successful example

```
nvidia-smi
```

There would be a GPU driver version and CUDA version show in the log.

NVIDIA-SMI 580.95.05				Driver Version: 580.95.05		CUDA Version: 13.0	
GPU Fan	Name Temp	Perf Perf	Persistence-M Pwr:Usage/Cap	Bus-Id	Disp.A Memory-Usage	Volatile GPU-Util	Uncorr. Compute M. MIG M.
0	NVIDIA RTX 4000 Ada Gene...	Off	00000000:16:00.0 Off				
30%	45C	P8	11W / 130W	1395MiB / 20475MiB	0%	Default	N/A
1	NVIDIA RTX 4000 Ada Gene...	Off	00000000:34:00.0 Off				
30%	53C	P2	42W / 130W	1395MiB / 20475MiB	100%	Default	N/A
2	NVIDIA RTX 4000 Ada Gene...	Off	00000000:52:00.0 Off				
30%	31C	P8	13W / 130W	15MiB / 20475MiB	0%	Default	N/A
3	NVIDIA RTX 4000 Ada Gene...	Off	00000000:70:00.0 Off				
30%	34C	P8	12W / 130W	15MiB / 20475MiB	0%	Default	N/A
Processes:							
GPU	GI	CI	PID	Type	Process name	GPU Memory	Usage
0	N/A	N/A	1961	G	/usr/lib/xorg/Xorg	4MiB	
0	N/A	N/A	3478799	C	/usr/bin/python3	1372MiB	
1	N/A	N/A	1961	G	/usr/lib/xorg/Xorg	4MiB	
1	N/A	N/A	3478800	C	/usr/bin/python3	1372MiB	
2	N/A	N/A	1961	G	/usr/lib/xorg/Xorg	4MiB	
3	N/A	N/A	1961	G	/usr/lib/xorg/Xorg	4MiB	

2.1.3. Install GPU Toolkit : CUDA

- For NVIDIA GPU

```
wget
https://developer.download.nvidia.com/compute/cuda/13.0.0/local_installers/cuda_13.0.0_580.65.06_linux.run
sudo sh cuda_13.0.0_580.65.06_linux.run
# Continue > accept > Look at the images below and do not check the box
# Turn [X] Driver --> [ ] Driver
# Then, move the cursor to Install and press Enter.
```

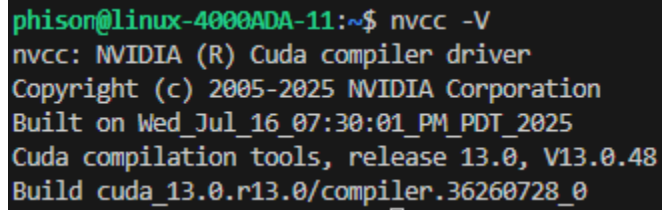
- Check CUDA version

- Export the environment variables

```
export CUDA_HOME=/usr/local/cuda-13.0
export PATH=$CUDA_HOME/bin:$PATH
export LD_LIBRARY_PATH=$CUDA_HOME/lib64:$LD_LIBRARY_PATH
```

- Check CUDA version: 13.0

```
nvcc -V
```



```
phison@linux-4000ADA-11:~$ nvcc -V
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2025 NVIDIA Corporation
Built on Wed Jul 16 07:30:01 PM PDT 2025
Cuda compilation tools, release 13.0, V13.0.48
Build cuda_13.0.r13.0/compiler.36260728_0
```

- Install NVIDIA Library: cuDNN

```
wget https://developer.download.nvidia.com/compute/cudnn/9.4.0/local_installers/cudnn-
local-repo-ubuntu2204-9.4.0_1.0-1_amd64.deb
sudo dpkg -i cudnn-local-repo-ubuntu2204-9.4.0_1.0-1_amd64.deb
sudo cp /var/cudnn-local-repo-ubuntu2204-9.4.0/cudnn-*-keyring.gpg
/usr/share/keyrings/
sudo apt-get update
sudo apt-get -y install cudnn-cuda-12
```

2.1.4. Install GPU Toolkit : cuDNN (9.17)

- For NVIDIA GPU (base on ubuntu version to install cuDNN)
 - <https://developer.nvidia.com/cudnn-9-17-0-download-archive>

2.2. Disk Setup (LVM Setting)

- Install LVM

```
sudo apt update;sudo apt install lvm2 xfsprogs.
```

- Check disk locations

```
lshw -class disk -class storage | grep -E 'ai100|logical name|version: EIFZ'
lsblk | grep nvme
```

```
# Confirm ai100 device names are, for example, nvme6n1 and nvme8n1. If not, make the
necessary changes below to adapt as appropriate.
```

- Clear disks just in case

```
sudo wipefs -a /dev/nvme1n1 /dev/nvme2n1
```

```
● phison@phison-X299-UD4-Pro:~$ sudo wipefs -a /dev/nvme1n1 /dev/nvme2n1
/dev/nvme1n1: 2 bytes were erased at offset 0x00000438 (ext4): 53 ef
```

- Create LVM

```
sudo pvcreate /dev/nvme1n1 /dev/nvme2n1
sudo vgcreate ai /dev/nvme1n1 /dev/nvme2n1
sudo lvcreate --type striped -i 2 -I 128k -l 100%FREE -n ai ai
```

```
● phison@phison-X299-UD4-Pro:~$ sudo pvcreate /dev/nvme1n1 /dev/nvme2n1
Physical volume "/dev/nvme1n1" successfully created.
Physical volume "/dev/nvme2n1" successfully created.
● phison@phison-X299-UD4-Pro:~$ sudo vgcreate ai /dev/nvme1n1 /dev/nvme2n1
Volume group "ai" successfully created
● phison@phison-X299-UD4-Pro:~$ sudo lvcreate --type striped -i 2 -I 128k -l 100%FREE -n ai ai
Logical volume "ai" created.
```

- Mount LVM

```
#Format the disk.
sudo mkfs.xfs -f -s size=4k -m crc=0 /dev/ai/ai -f

#Mount the disk.
sudo mkdir -p /mnt/nvme0
sudo mount /dev/ai/ai /mnt/nvme0
sudo chown -R $USER:$USER /mnt/nvme0
```

- (optional) Make mount persistent

```
# Make mount persistent
sudo echo '/dev/ai/ai /mnt/nvme0 xfs defaults,nofail 0 0' | sudo tee -a /etc/fstab

# Remove permanent mount setting
sudo sed -i '/dev/ai/ai/d' /etc/fstab
```

- Successful example

```
lsblk
```

If LVM setting is successful, you will see the following successful configuration when input “lsblk”.

```
● phison@phison-X299-UD4-Pro:~$ lsblk | grep -E 'nvme1n1|ai-ai|nvme2n1'
nvme1n1    259:0    0    1.9T    0 disk
└─ai-ai    252:0    0    3.6T    0 lvm   /mnt/nvme0
nvme2n1    259:1    0    1.8T    0 disk
└─ai-ai    252:0    0    3.6T    0 lvm   /mnt/nvme0
```

- (optional) If you need to dissolve LVM Setting

```
sudo umount /mnt/nvme0;sudo lvremove -y ai;sudo pvremove -y /dev/nvme1n1 /dev/nvme2n1  
--force --force
```

- If you only have one SSD, please follow the steps below to mount

```
sudo mkfs -t ext4 /dev/nvme1n1  
sudo mkdir -p /mnt/nvme0  
sudo mount /dev/nvme1n1 /mnt/nvme0  
sudo chown -R $USER:$USER /mnt/nvme0
```

PHISON Confidential

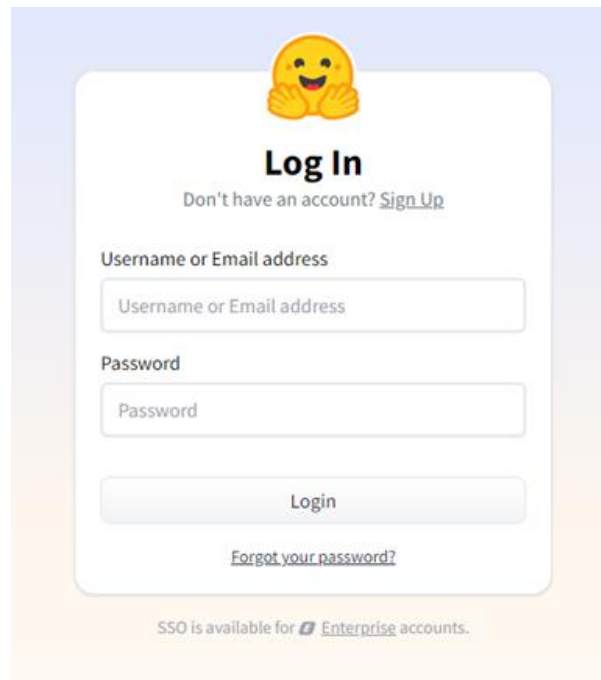
2.3. Login Huggingface

2.3.1. How to get Hugging Face token

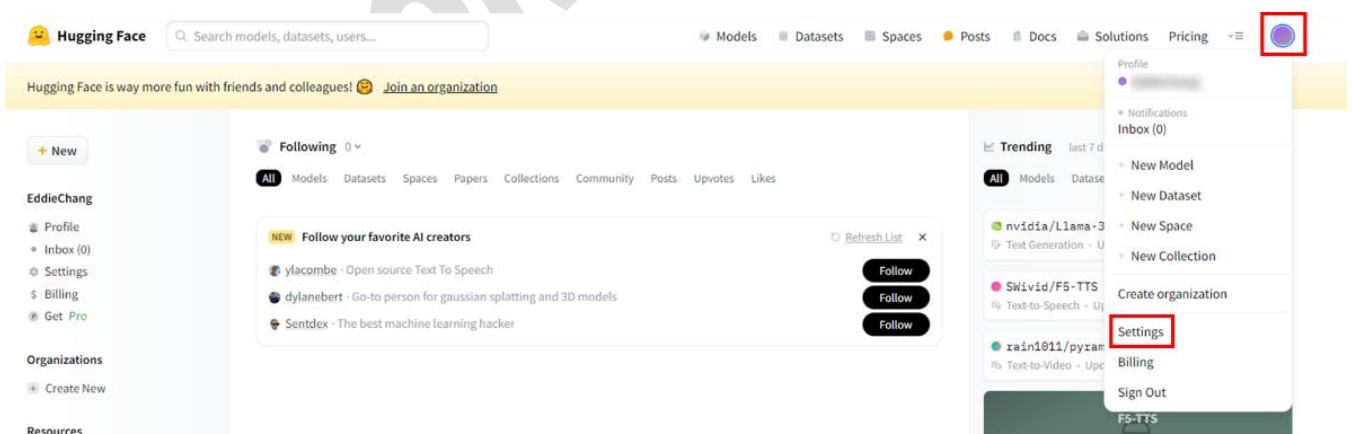
Setting your huggingface token in the environment. You can obtain your personal token at:

<https://huggingface.co/settings/tokens>

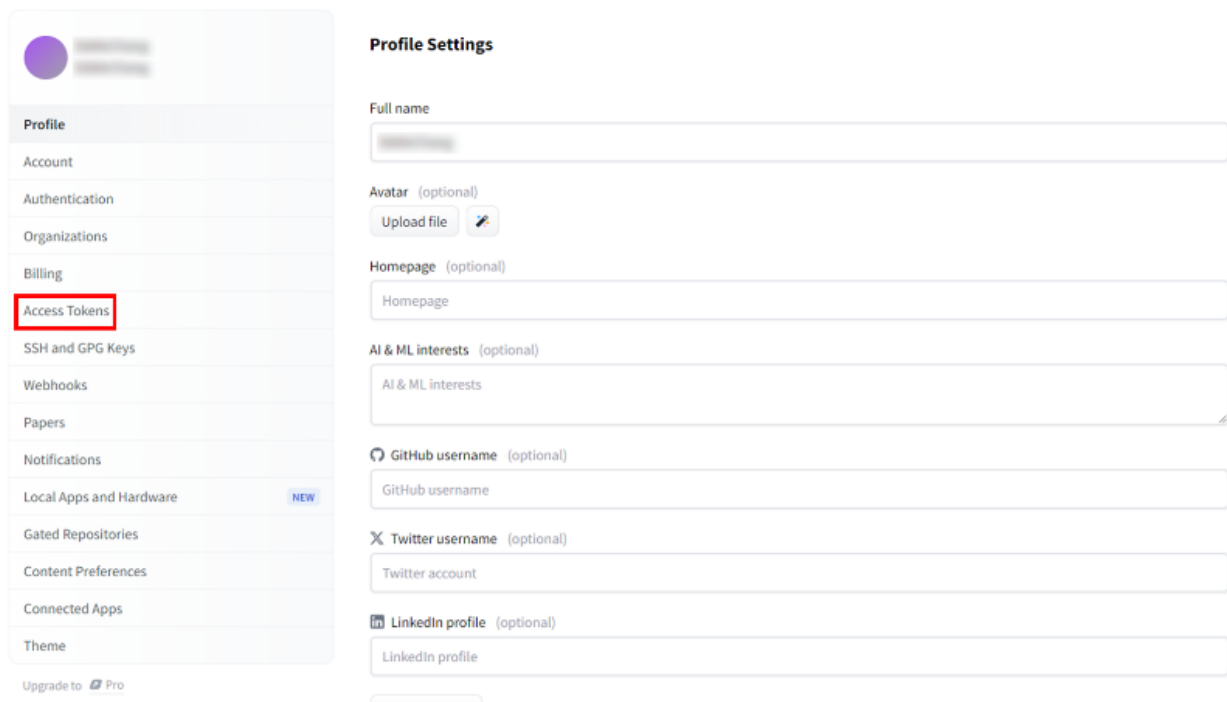
First, please register for a Hugging Face account. If you have already registered, please log in to Hugging Face directly.



After logging in, click on the account in the top right corner, open the menu, and select 'Settings'.

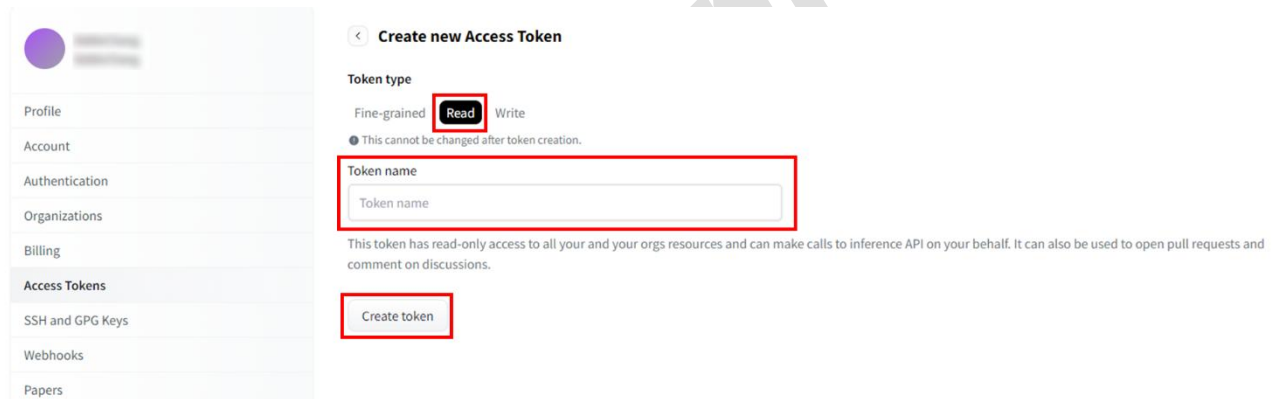


After entering 'Settings', click on 'Access Tokens' in the left column.



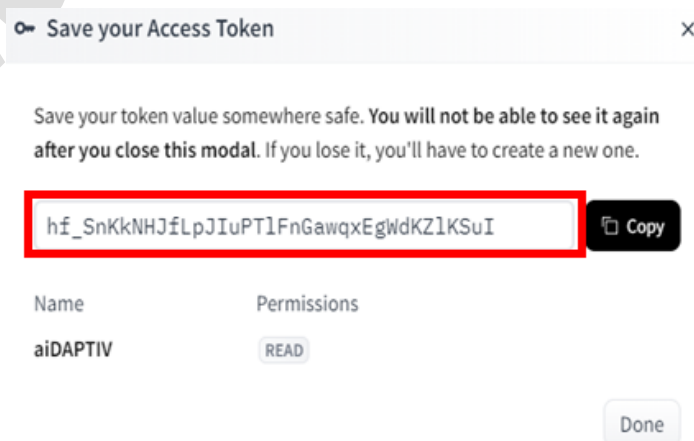
The screenshot shows the 'Profile Settings' page. On the left is a sidebar with various settings categories: Profile, Account, Authentication, Organizations, Billing, Access Tokens (highlighted with a red box), SSH and GPG Keys, Webhooks, Papers, Notifications, Local Apps and Hardware (marked with a 'NEW' tag), Gated Repositories, Content Preferences, Connected Apps, and Theme. The main area is titled 'Profile Settings' and contains several input fields for user information: Full name, Avatar (optional) with an 'Upload file' button, Homepage (optional), AI & ML interests (optional), GitHub username (optional), Twitter username (optional), and LinkedIn profile (optional).

Choose the 'Token type' and enter the 'Token name', then you can create the required token. You can make a selection based on the content and description of the 'Token type'. If you only need to download, choose 'Read'.



The screenshot shows the 'Create new Access Token' page. The left sidebar is the same as the previous screenshot, but 'Access Tokens' is now selected. The main area has a title 'Create new Access Token' and a 'Token type' section with two options: 'Fine-grained' (selected) and 'Read' (highlighted with a red box). Below this is a note: 'This cannot be changed after token creation.' The 'Token name' field is also highlighted with a red box. Below the field is a description: 'This token has read-only access to all your and your orgs resources and can make calls to inference API on your behalf. It can also be used to open pull requests and comment on discussions.' At the bottom, there is a 'Create token' button highlighted with a red box.

After choosing to create, a Hugging Face token will appear. This token is used for logging in later.



The screenshot shows a modal titled 'Save your Access Token' with a close button (X). The text inside says: 'Save your token value somewhere safe. You will not be able to see it again after you close this modal. If you lose it, you'll have to create a new one.' Below this text is a text input field containing the token value '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', which is highlighted with a red box. To the right of the field is a 'Copy' button. Below the token field is a table with two columns: 'Name' and 'Permissions'. The table has one row with 'aiDAPTIV' under 'Name' and 'READ' under 'Permissions'. At the bottom right of the modal is a 'Done' button.

Name	Permissions
aiDAPTIV	READ

The created token will be stored in the 'Access Token' in your personal account.

Name	Value	Last Refreshed Date	Last Used Date	Permissions
aiDAPTIV	hf_...KSul	less than a minute ago	-	READ

2.3.2. How to register the Hugging Face token

- Prerequisites

```
sudo apt update && sudo apt install git -y
pip install --upgrade huggingface_hub
```

- Authentication

```
# Configure Git to store credentials
git config --global credential.helper store

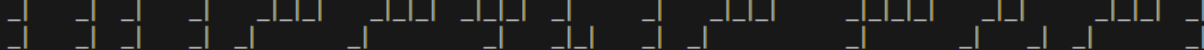
# Login to Hugging Face
huggingface-cli login

# Paste your <hf_token> from huggingface.co/settings/tokens when prompted
```

- Successful example

There would be a “Login successful” message

```
phison@phison-Pro-ET700I-W7:~/Desktop/aiDAPTIV2$ huggingface-cli login
```



```
To login, `huggingface_hub` requires a token generated from https://huggingface.co/settings/tokens .
Enter your token (input will not be visible):
Add token as git credential? (Y/n) y
Token is valid (permission: read).
Your token has been saved in your configured git credential helpers (store).
Your token has been saved to /home/phison/.cache/huggingface/token
Login successful
```

2.4. Download Llama-3.1-8B-Instruct

- Before downloading the Llama-3.1-8B-Instruct model, you need to request permission from huggingface.
 - <https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct>
 - It needs to wait for the license approval from huggingface

By agreeing you accept to share your contact information (email and username) with the repository authors.

First Name

Last Name

Date of birth

Country

Affiliation

Your country and region (based on approximate Internet address) will be shared with the model owner.

☐ By clicking Submit below I accept the terms of the license and acknowledge that the information I provide will be collected stored processed and shared in accordance with the Meta Privacy Policy

2.4.1. Download Llama-3.1-8B-Instruct model

```
mkdir -p /home/$USER/Desktop/llm
cd /home/$USER/Desktop/llm
mkdir Llama-3.1-8B-Instruct
huggingface-cli download --token HF_TOKEN --resume-download meta-llama/Llama-3.1-8B-Instruct \
--local-dir-use-symlinks False --local-dir Llama-3.1-8B-Instruct
# It would be take some time to download model.
```

- remark

HF_TOKEN: Please replace with <your_hf_token>

meta-llama/Llama-3.1-8B-Instruct: You can replace it with the model you want to download.

 [meta-llama/Llama-3.1-8B-Instruct](https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct) 

Llama-3.1-8B-Instruct: Please replace it with the folder you created

- Successful example
 - `mkdir -p Llama-3.1-8B-Instruct` in `/home/$USER/Desktop/llm/`

```
phison@phison-ASUS-ET700I-002:~/Desktop/llm$ mkdir Llama-3.1-8B-Instruct
```

```
phison@phison-ASUS-ET700I-002:~/Desktop/llm$ ls
Llama-3.1-8B-Instruct
```

- Download success message

```
original/params.json: 100% | 199/199 [00:00<00:00, 3.48MB/s]
Download complete. Moving file to Llama-3.1-8B-Instruct/original/params.json | 0.00/23.9k [00:00<?, ?B/s]
D
Downloading 'special_tokens_map.json' to 'Llama-3.1-8B-Instruct/.cache/huggingface/download/special_tokens_map.json.d8cd5076496dbe4be2320312abc10adc43097b81.incomplete'9MB/s]
tokenizer.model: 100% | 2.18M/2.18M [00:00<00:00, 17.9MB/s]
Download complete. Moving file to Llama-3.1-8B-Instruct/original/tokenizer.model
Downloading 'tokenizer.json' to 'Llama-3.1-8B-Instruct/.cache/huggingface/download/tokenizer.json.f916e71031fa08f3c6ef1680a590c15b52d3cdd9.incomplete'0/1.17G [00:00<?, ?B/s]
special_tokens_map.json: 100% | 73.0/73.0 [00:00<00:00, 1.10MB/s]
Download complete. Moving file to Llama-3.1-8B-Instruct/special_tokens_map.json
Downloading 'tokenizer_config.json' to 'Llama-3.1-8B-Instruct/.cache/huggingface/download/tokenizer_config.json.cb9ec2536e44d86778b10509d3e5bdca459a5cf.incomplete'18.1MB/s]
tokenizer_config.json: 100% | 50.5k/50.5k [00:00<00:00, 5.84MB/s]
Download complete. Moving file to Llama-3.1-8B-Instruct/tokenizer_config.json
tokenizer.json: 100% | 31.5M/4.98G [00:01<03:53, 21.2MB/s]
Download complete. Moving file to Llama-3.1-8B-Instruct/tokenizer.json
9.09M/9.09M [00:00<00:00, 10.7MB/s]
41.9M/4.98G [00:01<03:52, 21.3MB/s]
1.17G/1.17G [01:02<00:00, 18.7MB/s]
1.44G/4.98G [01:02<02:32, 23.2MB/s]
4.98G/4.98G [03:20<00:00, 24.8MB/s]
4.98G/4.98G [03:20<00:00, 29.7MB/s]
4.92G/4.92G [03:22<00:00, 24.3MB/s]
4.83G/4.92G [03:19<00:03, 24.7MB/s]
5.00G/5.00G [03:35<00:00, 23.2MB/s]
4.92G/4.92G [03:22<00:00, 31.1MB/s]
16.1G/16.1G [11:12<00:00, 23.9MB/s]
4.50G/16.1G [03:21<07:09, 25.9MB/s]
Download complete. Moving file to Llama-3.1-8B-Instruct/original/consolidated.00.pth | 17/17 [11:14<00:00, 39.68s/it]
Fetching 17 files: 100% |
/home/phison/Llama-3.1-8B-Instruct | 4.79G/16.1G [03:34<08:19, 22.6MB/s]
```

- The weight of the model will appear in "Llama-3.1-8B-Instruct"

```
phison@phison-ASUS-ET700I-002:~/Desktop/llm/Llama-3.1-8B-Instruct$ ls
config.json          model-00001-of-00004.safetensors  model-00004-of-00004.safetensors  README.md          tokenizer.json
generation_config.json  model-00002-of-00004.safetensors  model.safetensors.index.json      special_tokens_map.json  USE_POLICY.md
LICENSE              model-00003-of-00004.safetensors  original                          tokenizer_config.json
```

2.5. aiDAPTIV Middleware Installation

2.5.1. Local Depoly aiDAPTIV

Please use a fresh Ubuntu system to avoid environment conflicts. If not using a fresh system, please deploy the environment using Docker (2.5.2).]

Native Installation option (The deployed user can be a custom user or root.)

- setup tool (middleware)

```
wget https://phisonbucket.s3.ap-northeast-1.amazonaws.com/setup_vNXUN_2_05_A1.sh
```

- Deploy aiDAPTIV (Install middleware)
 - Execute setsetup_vNXUN_2_05_A1.sh

```
bash setup_vNXUN_2_05_A1.sh
#terminal output:
Hi user phison
Select an action:
1. Deploy aiDAPTIV+
2. Exit
Enter your choice (1, 2): 1 # Select "1. Deploy aiDAPTIV+"

```

- After Select "1. Deploy aiDAPTIV+"
 - Selecting 'Y', the firmware update process will be initiated.
 - Selecting 'N' will directly proceed with the installation.

```
#terminal output:
If the FW version does not match, do you need to update the SSD FW? (Y/N)
Please enter Y or N: N

```

- Here, select 'N' to proceed with the installation

```
#terminal output:
Hi user phison, would you like to download vNXUN_2_05_A1.tar from cloud? (Y/N)
Please enter Y or N:

```

- Select 'Y' to download the installation file from the cloud.
- Selecting 'N' to provide the installation file path from the local end.
- If you select 'N', please enter the path to the vNXUN_2_05_A1.tar file

```
#terminal output:
Hi user phison, would you like to download vNXUN_2_05_A1.tar from cloud? (Y/N)
Please enter Y or N: n
Please enter the path to the vNXUN_2_05_A1.tar file:

```

- If you can't get vNXUN_2_05_A1.tar from cloud, please enter the path to the vNXUN_2_05TA1.tar file

```
#terminal output:
Can't get vNXUN_2_05_A1.tar from cloud, Please enter the path to the
vNXUN_2_05_A1.tar file

```

Successful example

- It will show successful message in the log.

```
#terminal output:
Deploy Phison aiDAPTIV+ successfully, You MUST restart the session for the changes to
take effect.
Hi user phison
Select an action:
1. Deploy aiDAPTIV+
2. Exit
3. FW update
Enter your choice (1, 2, 3):
```

- There would be a “aiDAPTIV2” folder in ~/aiDAPTIV2

```
phison@phison-Z690-AERO-D:~/aiDAPTIV2$ ls
automatic-speech-recognition  customize_model_support  image-text-to-text  requirements_and_mi.txt  requirements_nv.txt  vllm
commands                     fill-mask                phison_comm        requirements_and_radeon.txt  text-generation
```

Check Middleware version:

```
phisonai2 -v
```

- example

```
#terminal output:
phisonai2: Phison aiDAPTIV aiDAPTIVLink
COPYRIGHT (C) 2024 Phison Electronics Corp.
version: vNXUN205_A1
```


2.5.2. Train model in Docker.

In this section, you will learn how to use docker to do your training. You can use GPU resource and aiDAPTIVCache in docker.

- Docker Installation option

- Docker official website :

- <https://docs.docker.com/engine/install/ubuntu/>

- Download NVIDIA Container Toolkit:

- <https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html>

```
# Restart docker service to adopt the change  
sudo systemctl restart docker
```

- Docker image

```
wget https://phisonbucket.s3.ap-northeast-1.amazonaws.com/aiDAPTIV_vNXUN_2_05_  
A1.tar.gz
```

- Load docker image

```
sudo docker load < aiDAPTIV_vNXUN_2_05_A1.tar.gz
```

```
#terminal output
```

```
9d592720ced4: Loading layer
```

```
[=====>] 80.63MB/80.63MB
```

```
8f24e95e4178: Loading layer
```

```
[=====>] 60.58MB/60.58MB
```

```
5072e0bd4014: Loading layer
```

```
[=====>] 25.96MB/25.96MB
```

```
bdf78b39dc31: Loading layer
```

```
[=====>] 22.53kB/22.53kB
```

```
313506180a5b: Loading layer
```

```
[=====>] 245.5MB/245.5MB
```

```
87bc371dd28e: Loading layer
```

```
[=====>] 103.6MB/103.6MB
```

```
94049d3cf397: Loading layer
```

```
[=====>] 12.6MB/12.6MB
```

```
3fadd7664181: Loading layer
```

```
[=====>] 25.98MB/25.98MB
```

```
555ed4b97924: Loading layer
```

```
[=====>] 9.369GB/9.369GB
```

```
cc2e49bd9927: Loading layer
```

```
[=====>] 4.096kB/4.096kB
```

```
Loaded image: aidaptiv:vNXUN_2_05_A1
```

- Check docker image list

```
sudo docker image list
```

```
#terminal output
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
aidaptiv	vNXUN_2_05_A1	b7fe35a75e49	3 days ago	9.86GB

- The commands folder within docker will be deployed at:

```
/home/root/aiDAPTIV2/commands
```

There will be two folders env_config, exp_config and example.sh. There will be env_config.yaml and exp_config.yaml in env_config and exp_config respectively, which can be modified directly.

```
--commands
| --env_config
|   |--env_config.yaml
| --exp_config
|   |--env_config.yaml
| --example.sh
```

- Run aiDAPTIV Image

If you are deploying the environment using Docker, please execute the following. If you are using a native environment, please ignore this item.

- docker and nvidia gpu runtime Installation

- <https://docs.docker.com/engine/install/>
- <https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html>

```
sudo docker run --gpus all -it --ipc=host --privileged=true --ulimit memlock=-1 \
--ulimit stack=67108864 -v </path/to/model>:/app -v </path/to/LVM>:/mnt \
-v /dev/mapper:/dev/mapper -v /var/lock:/var/lock aidaptiv:vNXUN_2_05_A1
```

- Successful example

```
#terminal output
phison@phison:~$sudo docker run --gpus all -it --ipc=host --privileged=true
--ulimit memlock=-1 --ulimit stack=67108864 -v /mnt/model/LLM:/app -v /mnt
/nvme0:/mnt -v /dev/mapper:/dev/mapper -v /var/lock:/var/lock aidaptiv:vNXU
N_2_05_A1
```

- How to update FW in docker

- Check FW version if need to update

- command:

```
sudo apt install nvme-cli
sudo nvme list #確認要升級的是哪個 nvme
python3 -c "from phisonlib.moirai import nvme_update_flow; nvme_update_flow
('<nvme_path>', 0)" # <nvme_path>替換成要升級的是哪個 nvme 路徑
```

- example:

```
root@8cc0ab9f8b97:/# sudo nvme list
Node          Generic          SN              Model              Namespace  Usage              Format              FW Rev
-----
/dev/nvme0n1  /dev/ng0n1        587F075A102A00000145 PASCARI A1802K032T00E008T1900 0x1        2.05 TB / 2.05 TB 512 B + 0 B EIFZ61.3
/dev/nvme1n1  /dev/ng1n1        587F0751172A00000083 PASCARI A1802K032T00E008T1900 0x1        1.02 TB / 1.02 TB 512 B + 0 B EIFZ61.5
root@8cc0ab9f8b97:/# python3 -c "from phisonlib.moirai import nvme_update_flow; nvme_update_flow('/dev/nvme0n1', 0)"
Firmware needs to be updated
root@8cc0ab9f8b97:/# sudo nvme list
Node          Generic          SN              Model              Namespace  Usage              Format              FW Rev
-----
/dev/nvme0n1  /dev/ng0n1        587F075A102A00000145 PASCARI A1802K032T00E008T1900 0x1        2.05 TB / 2.05 TB 512 B + 0 B EIFZ61.3
/dev/nvme1n1  /dev/ng1n1        587F0751172A00000083 PASCARI A1802K032T00E008T1900 0x1        1.02 TB / 1.02 TB 512 B + 0 B EIFZ61.5
```

- Update FW

- command:

```
sudo apt install nvme-cli
sudo nvme list #確認要升級的是哪個 nvme
python3 -c "from phisonlib.moirai import nvme_update_flow; nvme_update_flow
('<nvme_path>', 1)" # <nvme_path>替換成要升級的是哪個 nvme 路徑
```

- example:

```
root@8cc0ab9f8b97:/# sudo nvme list
Node          Generic          SN              Model              Namespace  Usage              Format              FW Rev
-----
/dev/nvme0n1  /dev/ng0n1        587F075A102A00000145 PASCARI A1802K032T00E008T1900 0x1        2.05 TB / 2.05 TB 512 B + 0 B EIFZ61.3
/dev/nvme1n1  /dev/ng1n1        587F0751172A00000083 PASCARI A1802K032T00E008T1900 0x1        1.02 TB / 1.02 TB 512 B + 0 B EIFZ61.5
root@8cc0ab9f8b97:/# python3 -c "from phisonlib.moirai import nvme_update_flow; nvme_update_flow('/dev/nvme0n1', 1)"
[INFO]Firmware of selected device is up to date
root@8cc0ab9f8b97:/# sudo nvme list
Node          Generic          SN              Model              Namespace  Usage              Format              FW Rev
-----
/dev/nvme0n1  /dev/ng0n1        587F075A102A00000145 PASCARI A1802K032T00E008T1900 0x1        2.05 TB / 2.05 TB 512 B + 0 B EIFZ61.5
/dev/nvme1n1  /dev/ng1n1        587F0751172A00000083 PASCARI A1802K032T00E008T1900 0x1        1.02 TB / 1.02 TB 512 B + 0 B EIFZ61.5
```

2.6. Run aiDAPTIV

“phisonai2” command is model training command.

Remind! It might take hours to finish the job. (Base on your dataset size and device performance.)

- Check nccl version

```
pip list | grep nvidia-nccl-cu12
# version should be 2.27.7
```

- Start training your model.

Training model “Llama-3-8B-Instruct” with dataset “Dahoas/rm-static”

```
phisonai2 --env_config <env_config.yaml path> --exp_config <exp_config.yaml path>
```

If you want to run the training with multiple Pro 6000 blackwall GPUs, please add the flage NCCL_P2P_DISABLE = 1 at the beggining of the script.

```
NCCL_P2P_DISABLE=1 phisonai2 --env_config <env_config.yaml path> --exp_config <exp_config.yaml path>
```

First, find the location where the model is stored.

```
phison@phison-ASUS-ET700I-002:~/Desktop/llm$ ls
Meta-Llama-3.1-8B-Instruct
```

Use the ‘lsblk’ command to confirm the location of the mounted SSD.

```
nvme1n1
├── 259:0    0    1.9T  0 disk
└── ai-ai
    ├── 252:0    0    3.7T  0 lvm /mnt/nvme0
    └── nvme0n1
        ├── 259:1    0    1.9T  0 disk
        └── ai-ai
            ├── 252:0    0    3.7T  0 lvm /mnt/nvme0
            └──
```

Write the required parameters into

```
‘/home/$USER/aiDAPTIV2/commands/env_config/env_config.yaml’
```

```
‘/home/$USER/aiDAPTIV2/commands/exp_config/exp_config.yaml’
```

or replace it with the path where you store ‘env_config.yaml’ and ‘exp_config.yaml’.

Remark:

1. Remember to replace \$USER with the current user.
2. For more efficient training, it is recommended to set 'triton' to true.

- text-generation settings

Example of env_config.yaml

```
# Save_path, nvme_path, log_name settings
path_settings:
  lora:
    lora_weight: ""      # whether to load lora_weight (only activated when lora: true)
    lora_output_dir: ""  # whether to save lora adapter weight (only activated when lora: true)
  model_name_or_path: "/home/$USER/Desktop/llm/Llama-3.1-8B-Instruct"
  train_data_path:
    # - ./dataset_config/image-text-to-text/VQA_dataset_config.yaml
    # - ./dataset_config/text-generation/Pretrain_dataset_config.yaml
    # - ./dataset_config/text-generation/RAG_dataset_config.yaml
    # - ./dataset_config/text-generation/Harmony_dataset_config.yaml
    - ./dataset_config/text-generation/QA_dataset_config.yaml
  val_data_path: null
    # - ./dataset_config/image-text-to-text/VQA_dataset_config.yaml
    # - ./dataset_config/text-generation/Pretrain_dataset_config.yaml
    # - ./dataset_config/text-generation/RAG_dataset_config.yaml
    # - ./dataset_config/text-generation/Harmony_dataset_config.yaml
    # - ./dataset_config/text-generation/QA_dataset_config.yaml
  nvme_path: "/mnt/nvme0"
  output_dir: "/home/$USER/output"
  log_name: "Llama-3.1-8B-Instruct.log"
```

```
process_settings:
  master_port: 8299
  num_gpus: 1
  specify_gpus: null
run_settings:
  task_type: "text-generation"
  task_mode: "train" # or "/home/$USER/aiDAPTIV2/text-generation/train.py"
  per_device_train_batch_size: 1
  per_update_total_batch_size: 4
  num_train_epochs: 1
  max_iter: -1
  max_seq_len: 2048
  triton: True
  weight_file_format: null
  from_config: false
  precision_mode: 1

model_saver:
  max_num_of_saved_model_on_epoch_end: -1 # If the value is -1, it is equal to num_
train_epochs.
  enable_save_model_on_iteration: false
  max_num_of_saved_model_on_iteration: 2
  num_of_iteration_to_save_model: 2

lr_scheduler:
  mode: -1
  learning_rate: 0.000007

optimizer:
  beta1: 0.9
  beta2: 0.95
  eps: 0.00000001
  weight_decay: 0.01

lora:
  enable_lora: false
  lora_rank: 8
  lora_alpha: 16
  lora_task_type: "CAUSAL_LM"
```

- Monitor training status
 - Open another window, the log will be generated inside the folder where the command is executed.

```
#Open another session
tail -f <your_log>
# example: tail -f Phison_2024_05-15_1200.log
```

- If the training logs are checked and the loss after updates shows a decreasing trend, it indicates a successful training.

```
[2024-09-20 10:38:35.375315] [PHISON START] Epoch: 0, Iteration: 6
[2024-09-20 10:38:35.375436] [Forward][start]
[2024-09-20 10:38:37.103003] [Forward][time spent]:1.7275550365447998
[2024-09-20 10:38:37.198474] [Loss]:2.2202885150909424
[2024-09-20 10:38:37.198491] [Backward][start]
[2024-09-20 10:38:58.795814] [Backward][time spent]:21.597309350967407
[2024-09-20 10:38:58.796005] [Update][Start]
[2024-09-20 10:38:59.218915] [Update][time spent]:0.42287421226501465
[2024-09-20 10:38:59.219000] [PHISON END] Iteration: 6

[2024-09-20 10:38:59.222052] [PHISON START] Epoch: 0, Iteration: 7
[2024-09-20 10:38:59.222171] [Forward][start]
[2024-09-20 10:39:00.944030] [Forward][time spent]:1.721846580505371
[2024-09-20 10:39:01.040680] [Loss]:2.3363866806030273
[2024-09-20 10:39:01.040695] [Backward][start]
[2024-09-20 10:39:10.869449] [Backward][time spent]:9.828744173049927
[2024-09-20 10:39:10.869523] [Update][Start]
[2024-09-20 10:39:35.695246] [Update][time spent]:24.825713872909546
[2024-09-20 10:39:35.695333] [PHISON END] Iteration: 7

Training efficiency: 198.45754094494418 (tokens/s)

[2024-09-20 10:39:35.696001] [Save Model_Checkpoint][Start]
[2024-09-20 10:39:53.636669] [Save Model_Checkpoint][time spent]:17.94063425064087
Beginning of Epoch 2/4, Total Micro Batches 38128
[2024-09-20 10:39:53.642974] [PHISON START] Epoch: 1, Iteration: 0
[2024-09-20 10:39:53.654870] [Forward][start]
[2024-09-20 10:39:55.376546] [Forward][time spent]:1.7216582298278809
[2024-09-20 10:39:55.472200] [Loss]:1.9732047319412231
[2024-09-20 10:39:55.472217] [Backward][start]
[2024-09-20 10:40:00.176516] [Backward][time spent]:4.704289197921753
[2024-09-20 10:40:00.176582] [Update][Start]
[2024-09-20 10:40:00.394558] [Update][time spent]:0.21796607971191406
[2024-09-20 10:40:00.394635] [PHISON END] Iteration: 0

[2024-09-20 10:40:00.397485] [PHISON START] Epoch: 1, Iteration: 1
[2024-09-20 10:40:00.397604] [Forward][start]
[2024-09-20 10:40:02.151702] [Forward][time spent]:1.7540862560272217
[2024-09-20 10:40:02.248785] [Loss]:1.5672987699508667
[2024-09-20 10:40:02.248797] [Backward][start]
[2024-09-20 10:40:10.331515] [Backward][time spent]:8.082708597183228
[2024-09-20 10:40:10.331587] [Update][Start]
[2024-09-20 10:40:10.711014] [Update][time spent]:0.37941741943359375
[2024-09-20 10:40:10.711099] [PHISON END] Iteration: 1
```


- Successful example

You get your first finetuned model in <output_dir> /home/\$USER/output !!

Finetuned model file: model-index.safetensors

```
phison@phison-ASUS-ET700I-002:~/output/finetuned_model_2024-09-20-10-36-37/epoch_3_step_7_Meta-Llama-3.1-8B-Instruct$ ls -l
total 15693120
-rw-rw-rw- 1 phison phison      951  九  20 10:46 config.json
-rw-rw-rw- 1 phison phison 5295466472  九  20 10:46 model-00001.safetensors
-rw-rw-rw- 1 phison phison 5352157840  九  20 10:46 model-00002.safetensors
-rw-rw-rw- 1 phison phison 4362258776  九  20 10:46 model-00003.safetensors
-rw-rw-rw- 1 phison phison 1050673280  九  20 10:46 model-00004.safetensors
-rw-rw-rw- 1 phison phison   22506  九  20 10:46 model.safetensors.index.json
-rw-rw-rw- 1 phison phison     345  九  20 10:46 special_tokens_map.json
-rw-rw-rw- 1 phison phison   50919  九  20 10:46 tokenizer_config.json
-rw-rw-rw- 1 phison phison  9084449  九  20 10:46 tokenizer.json
```

- image-text-to-text settings

Example of env_config.yaml

```
# Save_path, nvme_path, log_name settings
path_settings:
  lora:
    lora_weight: ""          # whether to load lora_weight (only activated when lora: true)
    lora_optimizer: ""       # whether to load lora_optimizer (only activated when lora: true)
    lora_output_dir: ""      # whether to save lora adapter weight (only activated when lora: true)
  model_name_or_path: "/home/$USER/Desktop/llm/Llama-3.2-11B-Vision-Instruct"
  multi_node_env_path: null
  optimizer_path: ""        # whether to load optimizer
  train_data_path:
    # - ./dataset_config/automatic-speech-recognition/dataset_config.yaml
    - ./dataset_config/image-text-to-text/VQA_dataset_config.yaml
    # - ./dataset_config/text-generation/Pretrain_dataset_config.yaml
    # - ./dataset_config/text-generation/RAG_dataset_config.yaml
    # - ./dataset_config/text-generation/Harmony_dataset_config.yaml
    # - ./dataset_config/text-generation/QA_dataset_config.yaml
  val_data_path: null
    # - ./dataset_config/automatic-speech-recognition/dataset_config.yaml
    # - ./dataset_config/image-text-to-text/VQA_dataset_config.yaml
    # - ./dataset_config/text-generation/Pretrain_dataset_config.yaml
    # - ./dataset_config/text-generation/RAG_dataset_config.yaml
    # - ./dataset_config/text-generation/QA_dataset_config.yaml
  nvme_path: "/mnt/nvme0"
  output_dir: "/home/$USER/output"
  log_name: "Llama-3.2-11B-Vision-Instruct.log"
```

Example of exp_config.yaml

```
process_settings:
  master_port: 8299
  num_gpus: 1
  specify_gpus: null
  master_port: 8299
  multi_node_settings:
    enable: False
    master_addr: "127.0.0.1"

run_settings:
  task_type: "image-text-to-text"
  task_mode: "train" # or "/home/$USER/aiDAPTIV2/text-generation/train.py"
  per_device_train_batch_size: 1
  per_update_total_batch_size: 4
  num_train_epochs: 1
  max_iter: -1
  max_seq_len: 2048
  triton: True
  weight_file_format: null
  from_config: false
  precision_mode: 1
  enable_save_optimizer_state: false

model_saver:
  max_num_of_saved_model_on_epoch_end: -1 # If the value is -1, it is equal to num_train_epochs.
  enable_save_model_on_iteration: false
  max_num_of_saved_model_on_iteration: 2
  num_of_iteration_to_save_model: 2

lr_scheduler:
  mode: -1
  learning_rate: 0.000007

optimizer:
  beta1: 0.9
  beta2: 0.95
  eps: 0.00000001
  weight_decay: 0.01

early_stop:
  enable: false
  min_delta: 0.01
  patience: 2
  verbose: false

lora:
  enable_lora: false
  lora_rank: 8
  lora_alpha: 16
  lora_task_type: "CAUSAL_LM"
  lora_target_modules: null
```

Using the following argument, you can complete your training task.

env_config.yaml

```
path_settings:
  lora:
    lora_weight: "str"          # whether to load lora_weight (only activated when lora:
                                true)
    lora_optimizer: "str"       # whether to load lora_optimizer (only activated when
                                lora: true)
    lora_output_dir: "str"      # whether to save lora adapter weight (only activated wh
                                en lora: true)
  model_name_or_path: "str"
  multi_node_env_path: null
  optimizer_path: "str" # whether to load optimizer
  train_data_path:
    - ./dataset_config/image-text-to-text/VQA_dataset_config.yaml
    # - ./dataset_config/text-generation/Pretrain_dataset_config.yaml
    # - ./dataset_config/text-generation/RAG_dataset_config.yaml
    # - ./dataset_config/text-generation/Harmony_dataset_config.yaml
    # - ./dataset_config/text-generation/QA_dataset_config.yaml
  val_data_path: null
    # - ./dataset_config/image-text-to-text/VQA_dataset_config.yaml
    # - ./dataset_config/text-generation/Pretrain_dataset_config.yaml
    # - ./dataset_config/text-generation/RAG_dataset_config.yaml
    # - ./dataset_config/text-generation/Harmony_dataset_config.yaml
    # - ./dataset_config/text-generation/QA_dataset_config.yaml
  nvme_path: "str"
  output_dir: "str"
  log_name: "str"
```

exp_config.yaml

```

process_settings:                                # <examples>
  num_gpus: 'int'                                # 1, 2, 4, 8
  specify_gpus: [null|'str']                     # null, '0,1,2,3', '4,5'
  master_port: 'int'                             # 8299
  multi_node_settings:
    enable: 'bool'                              # true, false
    master_addr: 'str'                          # "127.0.0.1"

run_settings:
  task_type 'str'                               # 'text-generation', 'automatic-speech-recognition', 'fill-mask', 'image-text-to-text'
  task_mode 'str'                               # 'train', or '/home/$USER/aiDAPTIV2/text-generation/train.py'
  per_device_train_batch_size 'int'             # 10
  per_update_total_batch_size 'int'             # 100 It must be set as a multiple of (num_gpus
* per_device_train_batch_size).
  num_train_epochs 'int'                        # 5
  max_iter 'int'                                # -1, 100
  max_seq_len 'int'                             # 2048
  triton 'bool'                                 # true, false
  weight_file_format [null|'str']               # null, 'bin', 'pt', 'safetensors'
  from_config 'bool'                           # true, false
  precision_mode 'int'                          # 0, 1
  enable_save_optimizer_state: 'bool'           # true, false

model_saver:
  max_num_of_saved_model_on_epoch_end: -1 # If the value is -1, it is equal to num_
train_epochs.
  enable_save_model_on_iteration: false
  max_num_of_saved_model_on_iteration: 2
  num_of_iteration_to_save_model: 2

lr_scheduler
  mode 'int'                                    # -1, 0, 1, 2, 3, 4, 5
  learning_rate 'float'                        # 0.000007

optimizer
  beta1 'float'                                # 0.9
  beta2 'float'                                # 0.95
  eps 'float'                                  # 0.00000001
  weight_decay 'float'                         # 0.01

early_stop:
  enable: 'bool'                               # true, false
  min_delta: 'float'                           # 0.01
  patience: 'int'                              # 2
  verbose: 'bool'                              # true, false

lora
  enable_lora 'bool'                           # true, false
  lora_rank 'int'                              # 8
  lora_alpha 'int'                             # 16
  lora_task_type 'str'                         # 'CAUSAL_LM'
  lora_target_modules: null,'str',list[str]    # null

```

Arguments Descriptions

- **path_setting:**
 - **lora:**
 - **lora_weight:** absolute path to the pretrained lora weight folder (**default: ""**).
 - **lora_optimizer:** absolute path to the lora optimizer (**default: ""**).
 - **lora_output_dir:** absolute path for saving lora model (default: None, lora model will only be saved if you provide lora_output_dir). (**default: ""**).
 - **model_name_or_path:** **[Necessary!]** absolute path to the pretrained weight folder (downloaded from huggingface) (**default: None**).
 - **multi_node_env_path:** Multi node env config (**default: Null**).
 - **optimizer_path:** Input the path of the optimizer state saved from the previous training session. The current training session will continue based on this optimizer state. (**default: None**).
 - **train_data_path:** When adding dataset, the yaml file under **/commands/env_configs** need to be configured.
 - Currently only support:
 - VQA datasets
 - RAG datasets
 - QA datasets
 - Pretrain datasets
 - Harmony datasets

for RAG, QA dataset, **single** QA pair or **multi-turn** chat can both be accept

After configured, the yaml file path should be added to **commands/env_config/env_config.yaml**

```
train_data_path:
- ./dataset_config/image-text-to-text/VQA_dataset_config.yaml
- ./dataset_config/text-generation/Pretrain_dataset_config.yaml
- ./dataset_config/text-generation/RAG_dataset_config.yaml
- ./dataset_config/text-generation/QA_dataset_config.yaml
- ./dataset_config/text-generation/Harmony_dataset_config.yaml
```

To run the default **env_config.yaml**, the execute path should locate at **aiDAPTIV2/commands**.

- **val_data_path:** absolute path to the dataset used to validate the training (**default: None**).
- **nvme_path:** **[Necessary!]** mounting point to aiDAPTIVCache (ex:/mnt/nvme0) (default: None).
- **output_dir:** absolute path for saving finetuned model (**default: None, finetuned model will only be saved if you provide output_dir**).
- **log_name:** absolute path for saving training log (**default: Phison_“currenttime”.log**).

- process_settings:
 - num_gpus: number of gpus to be utilized (default: 1).
 - specify_gpus: (optional) specify GPUs to use. If you want to use No3 and No2 GPU, set specify_gpus="3,2" (default: null).
 - master_port: port used by PyTorch distributed for communication during training (default: 8299).
 - multi_node_settings:
 - ◆ enable: trigger multi node training (default: False).
 - ◆ master_addr: Master node (rank 0)'s address, should be either the IP address or the hostname of node 0, for single node multi-proc training, the --master_addr can simply be 127.0.0.1 (default: 127.0.0.1).
- run_settings:
 - task_type: Choose a task type, the folder in ~/aiDAPTIV2. We only support "text-generation", "automatic-speech-recognition" and "fill-mask" in this version (default: text-generation). Check task type and model are in AVL (Appendix E Support Model list).
 - task_mode: We support "train" and absolute path to the execution python file, ex: "/home/\$USER/aiDAPTIV2/text-generation/train.py" (default: train).
 - per_device_train_batch_size: batch size in each GPU (default: 1).
 - per_update_total_batch_size: batch size for one update (default: 128).
remark : It must be set as a multiple of (num_gpus * per_device_train_batch_size).
 Ex: if you have 4 GPUs, each of them have 4 batches. you want to update the model every 80 batches. Then set the per_device_train_batch_size = 4, per_update_total_batch_size = 80. Your machine will run $80/4/4=5$ iterations and update once.
 - num_train_epochs: how many epoch you want to train your model (default: 1).
 - max_iter: (optional) early stop iteration before running entire epoch. Set -1: execute all iterations for each epoch, set 10: only execute 10 iterations for each epoch. (default: -1).
 - max_seq_len: sequence length you want to train your language model (default: 2048).
 - ◆ Model limitations: if you run bert model, you must set max_seq_len=512. For other models, please refer to the info on the huggingface's model page.
 - triton: (optional) trigger triton training procedure. We include some of techniques from Triton to improve training efficiency. Llama, Mistral, Mixtral are currently supported (default: True).

- `weight_file_format`: file format for loading pretrained model. We only support “safetensors”, “bin”, “pt”, “pth” and “None” in this version. (default: None, we will automatically search file format.).
- `from_config`: whether randomly init model weight. (default: False)
- `precision_mode`: determines what precision to train your model. 0 for “BF16” and 1 for “BF16, FP32 mixed”. (default: 1)
- `enable_save_optimizer_state`: Default is false, Indicates whether to save the current optimizer state to the system disk. If set to true, it means you want to save the optimizer state from this training session for use in the next training session.(default: False).
- `model_saver`
 - ◆ `max_num_of_saved_model_on_epoch_end`: Specifies the maximum number of models to be saved during epoch cycles. If the number of saved models exceeds this value during the training process, the first model saved in the epoch will be removed. Setting this value to -1 is equivalent to `num_train_epochs`, meaning that every model saved per epoch will be retained (default: -1).
 - ◆ `enable_save_model_on_iteration`: Specifies whether to save the model during iteration cycles (default: false).
 - ◆ `max_num_of_saved_model_on_iteration`: Specifies the maximum number of models to be saved during iteration cycles. The concept is similar to `max_num_of_saved_model_on_epoch_end`. The value must be greater than 0, and it is recommended to use at least 2 to avoid issues where models may be lost due to device problems during the saving process (default: 2).
 - ◆ `num_of_iteration_to_save_model`: Specifies how often a model should be saved per number of steps, and it must be a multiple of `args.gradient_accumulation_steps`. The value must be greater than 0 (default: 2).
- `lr_scheduler`
 - ◆ `learning_rate`: learning rate, be aware of this hyper-parameter. It can affect training result (default: 7e-6).
 - ◆ `mode`: choose a learning rate mode (default: 1).
 - `mode == -1`: LinearLR(optimizer, start_factor=1, total_iters=1)
 - `mode == 0`: LinearLR(optimizer, start_factor=0.5, total_iters=20)

- mode == 1: CosineAnnealingLR(optimizer, T_max=150)
- mode == 2: ExponentialLR(optimizer, gamma=0.99)
- mode == 3: MultiplicativeLR(optimizer, lr_lambda=lambda epoch: 0.95)
- mode == 4: StepLR(optimizer, step_size=30, gamma=0.1)
- mode == 5: MultiStepLR(optimizer, milestones=[30,80], gamma=0.1)
- optimizer
 - ◆ beta1: hyper-parameter for adam optimizer (default: 0.9).
 - ◆ beta2: hyper-parameter for adam optimizer (default: 0.95).
 - ◆ eps: hyper-parameter for adam optimizer (default:1e-8).
 - ◆ weight_decay: weight decay coefficient.(default:1e-2).
- optimizer
 - ◆ beta1: hyper-parameter for adam optimizer (default: 0.9).
 - ◆ beta2: hyper-parameter for adam optimizer (default: 0.95).
 - ◆ eps: hyper-parameter for adam optimizer (default:1e-8).
 - ◆ weight_decay: weight decay coefficient.(default:1e-2).
- early_stop
 - ◆ enable: trigger early stop (default: False).
 - ◆ min_delta: The current val_loss must be more than min_delta away from the best val_loss to be considered improved (default: 0.01).
 - ◆ patience: If val_loss does not improve and exceeds the patience times, early stopping will be triggered (default:2).
 - ◆ verbose: trigger to print loss.(default:False).
- lora
 - ◆ enable_lora: trigger lora training procedure (default: False).
 - ◆ lora_rank: dimension of the low-rank matrices for lora (default: 8).
 - ◆ lora_alpha: scaling factor of the weight matrices for lora (default: 16).
 - ◆ lora_task_type: training task type for lora. We only support "CAUSAL_LM" in this version (default: "CAUSAL_LM").
 - ◆ lora_target_modules: The names of the modules to apply the adapter to. If this is specified, only the modules with the specified names will be replaced (default: "null").

2.7. Quick Start for pytorch

2.7.1. Without aiDAPTIV+ Middleware

*** Regular training procedure ***

- Import optimizer

```
# Without aiDAPTIV+ Middleware
from torch.optim import Adam
```

- Create model instance and add model.parameters to optimizer

```
# Without aiDAPTIV+ Middleware
model = prepare_bf16_hf_model(model_name_or_path=MODEL_NAME_OR_PATH, tokenizer=tokenizer).to(device)
optimizer = Adam(model.parameters()), LEARNING_RATE)
```

2.7.2. With aiDAPTIV+ Middleware

*** Only Few steps to adapt with aiDAPTIV+***

- Import API from site-packages

```
# With aiDAPTIV+ Middleware
from phisonlib.moirai import initialize, save_model, MoiraiConfig
```

- Create model instance and call initialize. And we're done!

```
# With aiDAPTIV+ Middleware
model = prepare_bf16_hf_model_init_stream(model_name_or_path=MODEL_NAME_OR_PATH, tokenizer=tokenizer)
moirai_config = prepare_config()
model, optimizer = initialize(module=model, config=moirai_config)
```

3. BENCHMARK BY AIDAPTIV TOOLKIT (OPTIONAL)

In this section, you will learn how to use aiDAPTIV Toolkit to check you machine performance..

Suggest to do benchmark by aiDAPTIV Toolkit, if your HW configuration is different to AVL or you want to test max batch size in your machine.

You can refer 3.3 to setup your batch size.

After completing this section, you will know that aiDAPTIV Toolkit is a specific evaluation tool which to help user to findout the best betch size setting for finetuning.

3.1. aiDAPTIV Toolkit Installation flow

3.1.1. *aiDAPTIV Toolkit*

```
wget https://phisonbucket.s3.ap-northeast-1.amazonaws.com/toolkit\_v2.3.1\_python312.zip
```

3.1.2. *Unzip download file*

```
unzip toolkit_v2.3.1_python312.zip
```

3.1.3. *Change to folder*

```
cd toolkit_v2.3.1_python312.zip
```

3.1.4. *Deploy aiDAPTIV Toolkit and related library*

```
pip install -r requirements.txt
```

3.2. Start aiDAPTIV Toolkit

3.2.1. Modify training setting in toolkit_v2.3.1_python312/project.ini

You can follow default project.ini setting.

```
[ENV_setting]
# Specify the training GPU index
specify_gpu_index =0,1
# GPU number of model training
num_gpus=2
# Training model path in local
model_name_or_path=/home/$USER/Desktop/llm/Llama-3.1-8B-Instruct
# Path of aiDAPTIVCache
nvme_path=/mnt/nvme0
# Password of root
pwd=test

[Performance_test]
# Start batch size of performance test
start_bs=8
# End batch size of performance test
end_bs=100
# Sequence length while training LLM Model
seq_len=2048
# Expect training time in hour
training_hour=0.5
# Enable Triton
triton=True
```

3.2.2. Enter aiDAPTIV Toolkit folder

```
cd toolkit_v2.3.1_python312/Script/Model_Test
```

3.2.3. Run aiDAPTIV Toolkit Performance Test

```
python3 aidaptest_run.py --t 1
```

- * Type == 1 (--t 1): Test performance
- * Type == 2 (--t 2): Find max batchsize
- * Type == 3 (--t 3): Find max batchsize + Test performance

Successful example

aiDAPTIVTest would be stop while finish.

```
plot_dram_result=True
plot_vram_result=True
plot_loss_result=True
plot_efficiency_result=True
plot_timespent_result=True
```

```
plot_smartinfo_result=True
[INFO] - [Monitor step] - Plot Monitor Log: successful
[2024-10-08 10:59:02] - INFO - [Monitor step] - Plot Monitor Log: successful
[INFO] - Remove finetune model:
[2024-10-08 10:59:02] - INFO - Remove finetune model:
[INFO] - [Script step] - Remove checkpoint: successful
[2024-10-08 10:59:02] - INFO - [Script step] - Remove checkpoint: successful
```

- Result

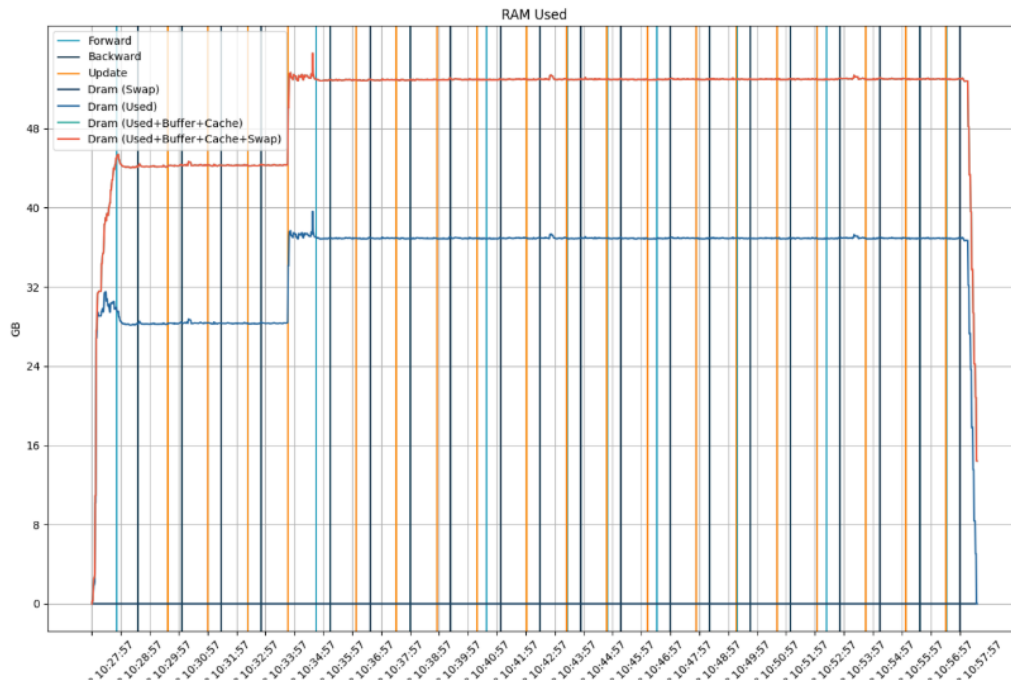
After using the aiDAPTIV Toolkit, the results will be stored in toolkit_v2.3.1_python312/Log according to the task, where you can check the execution results.

3.3. Performance Result

3.3.1. Log directory Description

1. Figure_4GPU_41bs (Model: Llama-3.1-8B-Instruct)

This folder will store figure of DRAM usage, GPU usage, Forward time, Backward time, Update time, training Loss and training speed.



2. Training log 4GPU_41bs (Model: Llama-3.1-8B-Instruct)

This log will store the output of terminal during aiDAPTIV training

```

41 Beginning of Epoch 1/1000, Total Micro Batches 61
42 [2024-10-08 10:28:49.499854] [PHISON START] Epoch: 0, Iteration: 0
43 [2024-10-08 10:28:49.510884] [Forward][start]
44 [2024-10-08 10:29:28.401107] [Forward][time spent]:38.89015984535217
45 [2024-10-08 10:29:32.831248] [Loss]:2.870795249938965
46 [2024-10-08 10:29:32.831346] [Backward][start]
47 [2024-10-08 10:30:35.113132] [Backward][time spent]:62.281774282455444
48 [2024-10-08 10:30:35.113201] [Update][Start]
49 [2024-10-08 10:30:35.446244] [Update][time spent]:0.33303284645080566
50 [2024-10-08 10:30:35.446299] [PHISON END] Iteration: 0

```

3. performance_result.xlsx (Model: Llama-3.1-8B-Instruct)

This Excel will organize the training information for various parameters in the aiDAPTIV Toolkit Performance test.

A	B	C	D	E	F
Test Model	GPU Used Num	Dataset	Seq_len	Gradient_accumulation_steps	
Meta-Llama-3.1-8B-Instruct	4	[../products/aiDAPTIVLink2048	4		
Batch size	1st Efficiency(token/s)	2nd Efficiency(token/s)	3rd Efficiency(token/s)	Avg Efficiency(2~3)(token/s)	10M Dataset training time
40	3129.31	3601.95	3780.97	3691.46	2708.96 seconds
41	3391.7	3611.48	3789.91	3700.695	2702.2 seconds

3.3.2. Performance

- The maximum batch size's average efficiency and the time required to train a 10M dataset can be found in the Performance Info sheet of performance_result.xlsx (see the red box in the figure below).
- Model: Llama-3.1-8B-Instruct (with triton)

A	B	C	D	E	F
Test Model	GPU Used Num	Dataset	Seq_len	Gradient_accumulation_steps	
Meta-Llama-3.1-8B-Instruct	4	[../products/aiDAPTIVLink2048	4		
Batch size	1st Efficiency(token/s)	2nd Efficiency(token/s)	3rd Efficiency(token/s)	Avg Efficiency(2~3)(token/s)	10M Dataset training time
40	3129.31	3601.95	3780.97	3691.46	2708.96 seconds
41	3391.7	3611.48	3789.91	3700.695	2702.2 seconds

- You can optimize hardware configuration to enhance training efficiency and reduce training dataset time.

3.3.3. Reference Data

- Following data just for reference. The data would affect by other device (ex: CPU/RAM...etc).
- The data sequence length = 2048 (with triton)

GPU	Model	Batchsize (per device)	max efficiency (token/s)
4090*4	Llama3-8B	52	6360
4090*4	Llama3-70B	23	631
5000ada*4	Llama3-8B	73	3182
5000ada*4	Llama3-70B	32	337
4000ada*4	Llama3-8B	44	3815
4000ada*4	Llama3-70B	19	376
4000ada*4	Llama3.1-8B	39	1917
4000ada*4	Llama3.1-8B	20	285

4. QA DATASETS

4.1. QA datasets

All QA datasets (local or from Huggingface) should be configured in “QA_dataset_config.yaml”.

- **Rules**

- Use - - - to separate multiple dataset
- Follow below format, where **DATASET NAME** must be different.

```
<DATASET NAME 1>:
```

```
  data_path:
  strategy: "qa"
  system_prompt:
  user_prompt:
  question_key:
  answer_key:
  exp_type:
  label_key:
```

```
- - -
```

```
<DATASET NAME 2>:
```

```
  .
  .
  .
```

- **Description for each value**

Key	Description
data_path	local path or huggingface dataset repository name
strategy	qa
system_prompt	system_prompt
user_prompt	user prompt should include {question} for question insertion (ex. user_prompt : solve the question {question})
question_key	key for question (NOT SUPPORT NESTED FORMAT)
answer_key	key for answer (NOT SUPPORT NESTED FORMAT)
exp_type	train
label_key	same as answer key

NOTE

If dataset is multiturn chat, the dataset should be converted to **question_key** : [question1, question2, question3, ...], **answer_key** : [answer1, answer2, answer3, ...]

4.2. Pretrain datasets

All Pretrain datasets (local or from Huggingface) should be configured in Pretrain_dataset_config.yaml

- **Rules**

- Use - - - to separate multiple dataset
- Follow following format, where **DATASET NAME** must be different

```
<DATASET NAME>:
  data_path:
  strategy: "pretrain"
  system_prompt:
  user_prompt:
  text:
  exp_type: ## train or inference or eval
- - -
<DATASET NAME>:
  .
  .
  .
```

- **Description for each value**

Key	Description
data_path	local path or huggingface dataset repository name
strategy	pretrain
system_prompt	system_prompt
user_prompt	user prompt should include {question} for question insertion (ex. user_prompt : solve the question {question})
text	key name for pretrain text (NOT SUPPORT NESTED FORMAT)
exp_type	train

4.3. RAG datasets

All RAG datasets (local or from Huggingface) should be configured in “RAG_dataset_config.yaml”.

- **Rules**

- Use - - - to separate multiple dataset
- follow the format, where **DATASET NAME** must be different

```
<DATASET NAME>:
  data_path:
  strategy: "rag"
  system_prompt:
  user_prompt:
  question_key:
  answer_key:
  rag_key:
  exp_type:
  label_key:
```

- - -

```
<DATASET NAME>:
```

```
.
.
.
```

- Description for each value

Key	Description
data_path	local path or huggingface dataset repository name
strategy	rag
system_prompt	[optional] system_prompt
user_prompt	user prompt should include {question} for question insertion and {rag} for rag data insertion (ex. Context: {rag}, based on the context answer the question : {question})
question_key	key for question
answer_key	key for answer
rag_key	key for rag data, (SUPPORT NESTED FORMAT) **
exp_type	train

Key	Description
label_key	same as answer key

```
** if RAG data has format
{context : {sentences : [RAG DATA]}}
the rag_key can be given

rag_key:
- context
- sentences
```

NOTE

if dataset is multiturn chat, the dataset should be convert to **question_key** : [question1, question2, question3, ...], **answer_key** : [answer1, answer2, answer3, ...]

4.4. Harmony datasets

- All Harmony datasets (local or from Huggingface) should be configured in Harmony_dataset_config.yaml

Rules

- use — to separate multiple dataset
- follow the format, where **DATASET NAME** must be different

```
<DATASET NAME>:
```

```
data_path:
```

```
strategy: "harmony"
```

```
exp_type:
```

```
message_key:
```

```
---
```

```
<DATASET NAME>:
```

```
.
```

```
.
```

```
.
```

- Description for each value

Key	Description
data_path	local path or huggingface dataset repository name
strategy	harmony
exp_type	train
message_key	list of role/content pairs that define the conversation (see notes below)

- Notes on format

Harmony defines conversations as a sequence of **messages**, where each message is associated with a role and may optionally carry a **channel** tag.

○ Roles

role	description
system	Used to provide high-level context or instructions to the model (knowledge, rules, constraints).
developer	Developer-specified guidance, often similar to system but intended for tool usage or more technical control.
user	Represents the input from the end user.
assistant	Output generated by the model, which can include tool calls or natural language replies.
tool	Information exchanged with tools, representing inputs/outputs related to tool usage.

○ Channels

channel	description
final	Messages that represent the assistant's final response intended for the end-user.
analysis	Internal reasoning steps (chain-of-thought) used by the model. Not normally exposed to the user.
commentary	Optional intermediate notes or commentary (e.g., tool call explanations), may accompany analysis.

○ Messages

- ◆ In Harmony datasets, all content is stored under a single key: **messages**.
- ◆ **messages** is a list of structured items; each item has at least:
 - role (as defined above)
 - content (the associated text)
- ◆ Channel information, if present, is embedded within the message entries.
- ◆ Not all roles or channels are required in every dataset sample. For example, some samples may only contain {user, assistant}, while others may also include developer or analysis.

4.5. example of datasets

4.5.1. Example 1 (QA dataset)

```
{
  "question": ".....",
  "cot_answer": "....."
},
<DATASET NAME>:
  data_path:
  strategy: "qa"
  system_prompt: "A chat between a curious user and an artificial intelligence
assistant. The assistant gives helpful, detailed, and polite answers to the user's
questions."
  user_prompt: answer the following question {question}
  question_key: question
  answer_key: cot_answer
  exp_type:
  label_key: cot_answer ## same as answer key
```

4.5.2. Example 2 (RAG dataset)

```
{
  "question": [
    "....",
    "....",
    "....."
  ],
  "answer": [
    "....",
    "....",
    "....."
  ],
  "hybrid_chunks": [
    "....",
    "....",
    "...."
  ]
},
```

<DATASET NAME>:

```
data_path:
strategy: "rag"
system_prompt:
user_prompt: you are a doctor with this background knowledge {rag}, now solve the
following question {question}
question_key: question
answer_key: answer
rag_key:
- hybrid_chunks
exp_type:
label_key: answer
```

4.5.3. Example 3 (nested RAG dataset)

```
"question": "...",
"cot_answer": "...",
"context": {
  "sentences": [
    "....."
  ]
}
```

<DATASET NAME>:

```
data_path:
strategy: "rag"
system_prompt:
user_prompt: you are a doctor with this background knowledge {rag}, now solve the
following question {question}
question_key: question
answer_key: answer
rag_key:
- context
- sentence
- 0
exp_type:
label_key: answer
```

4.5.4. example 4 (Harmony dataset)

```
{
  "messages": [
    {
      "content": "....",
      "role": "developer"
    },
    {
      "content": "....",
      "role": "user"
    },
    {
      "content": "....",
      "role": "assistant"
    }
  ]
}
```

<DATASET NAME>:

```
data_path:
strategy: "harmony"
exp_type:
message_key:
```

<DATASET NAME>:

```
.
.
.
```


4.6. Compare Change

origin method seen rm-static as pretrain data, hence no system and user prompt and template

4.6.1. Previous dataloader on Dahoas/rm-static after apply template

Human: How can I cook delicata squash?

Assistant: Try cutting it in half, cutting away the seeds and stringy parts, and laying it flat in a 400°F oven for 1 hour. You can drizzle it with olive oil and salt before you roast it if you want.

Human: Can you fry it?

Assistant: Try taking the squash halves you've roasted, and frying them in oil at a medium heat until it begins to brown and crisp on all sides. Taste it to see if it needs salt or spices.<|eot_id|>

PHISON.COM

4.6.2. Current dataloader on Dahoas/rm-static after apply template

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>
```

This is a chat between a user and an artificial intelligence assistant. The assistant gives helpful, detailed, and polite answers to the user's questions based on the context. The assistant should also indicate when the answer cannot be found in the context.<|eot_id|><|start_header_id|>user<|end_header_id|>

solve the question below :

Human: How can I cook delicata squash?

Assistant: Try cutting it in half, cutting away the seeds and stringy parts, and laying it flat in a 400°F oven for 1 hour. You can drizzle it with olive oil and salt before you roast it if you want.

Human: Can you fry it?

Assistant:<|eot_id|><|start_header_id|>assistant<|end_header_id|>

Try taking the squash halves you've roasted, and frying them in oil at a medium heat until it begins to brown and crisp on all sides. Taste it to see if it needs salt or spices.<|eot_id|><|start_header_id|>assistant<|end_header_id|>

4.6.3. Harmony dataset (HuggingFaceH4/Multilingual-Thinking) before applying template

```
{
  "messages": [
    {
      "content": "reasoning language: French\n\nYou are an AI chatbot with a lively and energetic personality.",
      "role": "developer"
    },
    {
      "content": "Can you show me the latest trends on Twitter right now?",
      "role": "user"
    },
    {
      "content": "Hey there! While I can't check Twitter (X) in real-time or access live data, I can share some...",
      "role": "assistant"
    }
  ]
}
```

4.6.4. Harmony dataset (HuggingFaceH4/Multilingual-Thinking) after applying Harmony template

```
<|start|>system<|message|>You are ChatGPT, a large language model trained by OpenAI.
```

```
Knowledge cutoff: 2024-06
```

```
Current date: 2025-08-29
```

```
Reasoning: medium
```

```
# Valid channels: analysis, commentary, final. Channel must be included for every message.<|end|>
```

```
<|start|>developer<|message|>reasoning language: French
```

```
You are an AI chatbot with a lively and energetic personality.<|end|>
```

```
<|start|>user<|message|>Can you show me the latest trends on Twitter right now?<|end|>
```

```
<|start|>assistant<|channel|>final<|message|>Hey there! While I can't check Twitter (X) in real-time or access live data, I can share some...<|end|>
```

5. MULTIMODALITY DATASET CONFIG

1. When adding dataset, the yaml file under /commands/image-text-to-text/env_configs need to be configured
2. Currently only support:
 - VQA datasets : QA dataset with vision
3. After configured, the yaml file path should be added to commands/env_config/env_config.yaml ``


```
yaml = data_path:
    ○ ./dataset_config/image-text-to-text/VQA_dataset_config.yaml``
```

 - to run the default env_config.yaml, the execute path should locate at aiDAPTIV2/commands

5.1. VQA datasets

All VQA datasets (local or from Huggingface) should be configured in image-text-to-text/VQA_dataset_config.yaml

- **Rules**
 - Use - - - to separate multiple dataset
 - Follow following format, where **DATASET NAME** must be different

```
<DATASET NAME>:

data_path:

image_folder : ""

strategy: "qa"

system_prompt: ""

user_prompt: "{question}"

question_key: ""

answer_key: ""

image_key : ""

label_key: ""

- - -

<DATASET NAME>:
```

5.2. Description for each value

Key	Description
data_path	local path / local folder or huggingface dataset repository
image_folder	The image folder for the custom dataset will be concatenated with the path within the dataset.
strategy	qa
system_prompt	system_prompt
user_prompt	user prompt should include {question} for question insertion (ex. user_prompt : solve the question {question})
question_key	key for question
rag_key	key for rag data, (SUPPORT NESTED FORMAT) **
answer_key	key for answer
image_key	The key for the image content can be a URL, local file path, or a PIL Image. Leave this field blank for text-only datasets.
label_key	same as answer key

5.3. Supported dataset

1. Repository from huggingface: With given repository name we support automatically download from huggingface. (ex. Multimodal-Fatima/OK-VQA_train)
2. Local huggingface dataset: If the dataset is cloned or downloaded from HuggingFace and comprises *.parquet files, designate the root folder as the data_path for subsequent steps.
 - Ex. for following Directory structure, the data_path should be Desktop/OK-VQA_train

Desktop/OK-VQA_train

```

|- data
  |- train-0000-of-0004.parquet
  |- train-0001-of-0004.parquet
  |-
  |-
|-README.md

```

5.3.1. Custom dataset: The data path should be in the format of `./json` or `./csv**` to properly parse the data

- Dataset_config example (QA dataset)

For example, utilize AI4Math/MathVista in HuggingFace for demonstration. The header include [pid, question, image, decode_image, choice, unit, precision, answer, ...]

The Dataset configuration can be set up as follows without the need to download the entire dataset locally.

5.3.2. Example (MathVista QA dataset)

<DATASET NAME>:

data_path: "AI4Math/MathVista"

image_folder : ""

strategy: "qa"

system_prompt: "you are a helpful assistant"

user_prompt: "Please address the following question using the accompanying image.

{question}"

question_key: "question"

answer_key: "answer"

image_key : "decode_image"

label_key: ""

5.3.3. dataset example

- The header in train.json should include [question, decode_image, answer] at least.

```
// train.json
[
  {
    "id": data_id,
    "question": question_to_ask,
    "answer": "answer1_to_reponse",
    "image": image_path,
    .
    .
    .
  },
  .
  .
  .
]
```

- **data_id** is the id of the data
- **question_to_ask** is the question
- **answer_to_reponse** is the answer of the data.
- **image_path** is the path of the image

5.3.4. dataset structure example

```
Desktop/custom_dataset
|- train.json
|- image_folder
    |- image1
    |- image2
    |- 
    |- 
|- README.md
```

- Only support .json or .csv for Custom dataset

5.3.5. Custom dataset config example

```
<DATASET NAME>:
data_path: "Desktop/custom_dataset/train.json"
image_folder : "Desktop/custom_dataset/image_folder"
strategy: "qa"
system_prompt: "The prompt you want before the training data."
user_prompt: "The prompt with the question {question}"
question_key: "question"
answer_key: "answer"
image_key : "image"
label_key: "label"
---
<DATASET NAME>:
```

Note :: The data with system_prompt and user_prompt used for training would becomes :

```
"The prompt you want before the training data."
image in dataset
"The prompt with the question {question in dataset}"
```

APPENDIX A. HOW TO SET SWAP FILE

- Enable swapping provides extra memory for DRAM. This can extend the range of batch size that you can use if you still have enough memory on the GPU.

```
# Create swap file
sudo dd if=/dev/zero of=/mnt/nvme0/swapfile bs=1M count=256k
# Modify permission
sudo chmod 0600 /mnt/nvme0/swapfile
# Initialize swap file
sudo mkswap /mnt/nvme0/swapfile
# Enable the swap
sudo swapon /mnt/nvme0/swapfile
# Make the swap permanent
sudo echo '/mnt/nvme0/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

- If you would like to remove the swap or unplug aiDAPTIVCache, please make sure to follow the steps below to prevent unexpected system issues.

```
# Disable the swap
sudo swapoff /mnt/nvme0/swapfile
# Remove permanent swap setting
sudo sed -i '/\mnt\/nvme0\/swapfile/d' /etc/fstab
# Remove swapfile (optional)
sudo rm /mnt/nvme0/swapfile
```

APPENDIX B. APPROVED VENDOR LIST (AVL)

B.1 GPU AVL

Vendor	Product Name
NVIDIA	H200
NVIDIA	H100
NVIDIA	RTX A6000
NVIDIA	RTX 6000 Ada
NVIDIA	RTX 5000 Ada
NVIDIA	RTX 4000 Ada
NVIDIA	L40/L40S
NVIDIA	RTX 50 series
NVIDIA	RTX 40 series
NVIDIA	RTX Pro 6000

B.2 CPU AVL

Brand	Naming
Intel	Xeon Gold 5320
Intel	Xeon Gold 6330
Intel	Xeon w5-3425
Intel	Xeon Gold 6538Y+
Intel	Xeon Silver 4410Y
Intel	Xeon Silver 4410T
Intel	Xeon Silver 5315Y
Intel	i9-13900
Intel	i9-12900E

AMD	Ryzen Threadripper 7980X 64-Cores
AMD	EPYC 7713P
AMD	EPYC 9174F

B.3 Support Model list

No	Task Type	Model Name
1	Text-Generation	Llama-3-Taiwan-70B-Instruct
2	Text-Generation	Llama_3.1_8B_Instruct
3	Text-Generation	Llama_3.1_70B_Instruct
4	Text-Generation	Llama_3.2_3B_Instruct
5	Image-Text-to-Text	Llama_3.2_11B_Vision_Instruct
6	Text-Generation	Llama_3.3_70B_Instruct
7	Text-Generation	Qwen2.5_7B_Instruct
8	Text-Generation	Qwen2.5_32B_Instruct
9	Text-Generation	Qwen2.5_72B_Instruct
10	Text-Generation	Qwen3-14B
11	Text-Generation	Qwen3_32B
12	Text-Generation	QwQ_32B
13	Text-Generation	DeepSeek_R1_Distill_Llama_70B
14	Text-Generation	DeepSeek_R1_Distill_Qwen_32B
15	Text-Generation	DeepSeek-R1-Distill-Qwen-7B
16	Image-Text-to-Text	gemma_3_27b_it
17	Text-Generation	Phi_4
18	Text-Generation	gpt_oss_20b
19	Text-Generation	gpt_oss_120b
20	Text-Generation	Mistral_Small_3.1_24B_Instruct_2503
21	Automatic-Speech-Recognition	whisper_large_v3

APPENDIX C. FW UPDATE

If fw update is not performed during installation, you can use `setup_vNXUN_2_05_A1.sh` to do fw check/update.

If aiDaptiv+ is successfully installed is correct, a third option “FW update” will appear.

- Select “3. FW update”

```
Deploy Phison aiDAPTIV+ successfully, You MUST restart the session for the changes to take effect.
Hi user phison
Select an action:
1. Deploy aiDAPTIV+
2. Exit
3. FW update
Enter your choice (1, 2 or 3):
```

- You can choose to FW check/update

```
Choose an action:
1) Check FW version
2) Update FW version
3) Back
4) Exit
Enter your choice (1, 2, or 3):
```

- After selecting check/update, select 1 to add the device that needs to perform fw check/update

```
Choose an action:
1) Add nvme_path
2) Terminate adding nvme_path
Please select an action: 1
Enter nvme_path to add: /dev/nvme0n1
```

- The added device will be displayed at the top. If you make a mistake in adding, select 2 to remove it. If you are done adding, select 3 to continue.

```
Current nvme_path list:
1) /dev/nvme0n1
Choose an action:
1) Add nvme_path
2) Remove nvme_path
3) Terminate adding nvme_path
Please select an action:
```

- Confirm again whether the added device is correct

```
Double-check the nvme_path list:
Current nvme_path list:
1) /dev/nvme0n1
Choose an action:
1) Confirm
2) Back
Please select an action:
```

- Successful example

- FW check

```
Double-check the nvme_path list:
Current nvme_path list:
1) /dev/nvme0n1
Choose an action:
1) Confirm
2) Back
Please select an action: 1
Confirming and checking...
Checking firmware for: /dev/nvme0n1
check_fw_version
[PHISON AIDAPTIV][INFO] Firmware of selected device needs to be updated
```

- FW update

```
Double-check the nvme_path list:
Current nvme_path list:
1) /dev/nvme0n1
Choose an action:
1) Confirm
2) Back
Please select an action: 1
Confirming and Updating...
Updating firmware for: /dev/nvme0n1
update_fw_version
[PHISON AIDAPTIV][INFO] Firmware of selected device is up to date
```

Q&A EN

- Q1. If I want to change the directory for downloading the LLAMA 2 Model, do I just need to modify the download directory path in setup.sh?
 - You can directly modify the path in setup.sh, but it is recommended to download first according to the original setup.sh and then move it. Remember to also change the model path in the phisonai2 parameter Model path after modification.
- Q2. How can I find out the number of tokens in datasets on HuggingFace? Is there a way to calculate the number of tokens?
 - The number of tokens in one training epoch = number of data rows * token length (default is 2048).
 - Taking the Hugging Face open dataset rm-static as an example, the number of tokens in one epoch is $76300 * 2048 = 156M$ (tokens).
- Q3. I Can all datasets on HuggingFace be used?
 - In the source code's utils/data/data_utils.py, the get_raw_dataset function lists datasets. We have only tested Dahoas/rm-static, others can be tried but we cannot guarantee successful training.
- Q4. How can I view the test data and results?
 - During training, the efficiency of training will be logged, showing tokens/s, which indicates how many tokens are trained per second on the given device conditions. The trained model will output to the specified output directory.
- Q5. The training process has not completely terminated. Is this normal?

```
saving the final model at epoch_0_step_19063 ...
[2024-02-21 23:40:10,169] [INFO] [engine.py:2924:save_10bit_model] Saving model weights to /media/user/nvme0/result/finetuned_model_2024-02-16-14-34-52/epoch_0_step_19063_#Llama-2-7b-hf/pytorch_model.bin, tag: global_step595
[2024-02-21 23:40:10,169] [INFO] [torch_checkpoint_engine.py:21:save] [Torch] Saving /media/user/nvme0/result/finetuned_model_2024-02-16-14-34-52/epoch_0_step_19063_#Llama-2-7b-hf/pytorch_model.bin...
[2024-02-21 23:40:10,179] [INFO] [torch_checkpoint_engine.py:33:commit] [Torch] Checkpoint global_step595 is ready now!
[2024-02-21 23:40:30,804] [INFO] [torch_checkpoint_engine.py:23:save] [Torch] Saved /media/user/nvme0/result/finetuned_model_2024-02-16-14-34-52/epoch_0_step_19063_#Llama-2-7b-hf/pytorch_model.bin.
[2024-02-21 23:40:30,804] [INFO] [torch_checkpoint_engine.py:33:commit] [Torch] Checkpoint global_step595 is ready now!
[2024-02-21 23:40:56,119] [INFO] [launch.py:350:main] Process 22032 exits successfully.
[2024-02-21 23:40:59,124] [INFO] [launch.py:350:main] Process 22031 exits successfully.
^[[B^[[B
```

- You can use tail -f filename to view a log file. Press Ctrl+C to exit. (Detailed explanation of the tail command)
- Q6. I want to use my custom Dataset.
 - Create a test.json file with the following format and input it into the --data_path parameter.

○

```
[
  {
    "instruct": "input1",
    "output": "output1"
  },
  {
    "instruct": "input2",
    "output": "output2"
  },
  {
    "instruct": "input3",
    "output": "output3"
  }
]
```

- Q7. Supported 33B Model

- <https://huggingface.co/lmsys/vicuna-33b-v1.3>

- Q8. Is Training Efficiency normal?

- Provide the following information:

- ◆ GPU model, quantity
- ◆ Number of SSDs in Raid0
- ◆ DRAM Size
- ◆ CPU model, quantity
- ◆ Executed Command parameters

- Q9. Will the speed of the disk where the model is stored affect the training speed?

- no

- Q10. Flash_attn undefined symbol

○

```
(myenv2) root@QX-phison:~/Desktop# /root/anaconda3/envs/myenv2/bin/python
Python 3.8.18 (default, Sep 11 2023, 13:40:15)
[GCC 11.2.0] :: Anaconda, Inc. on linux
Type "help", "copyright", "credits" or "license()" for more information.
>>> import flash_attn
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/root/anaconda3/envs/myenv2/lib/python3.8/site-packages/flash_attn/_init_.py", line 3, in <module>
    from flash_attn.flash_attn_interface import (
  File "/root/anaconda3/envs/myenv2/lib/python3.8/site-packages/flash_attn/flash_attn_interface.py", line 10, in <module>
    import flash_attn_2_cuda as flash_attn_cuda
ImportError: /root/anaconda3/envs/myenv2/lib/python3.8/site-packages/flash_attn_2_cuda.cpython-38-x86_64-linux-gnu.so: undefined symbol: _ZN2at4opsZeros4callIDC186ArrayRefINS2_65mIntEENS2_8optionalIN
S2_10scalarTypeEENS2_INS2_6layoutEENS2_INS2_6deviceEENS2_13vec
```

- Please refer to https://phisonbucket.s3.ap-northeast-1.amazonaws.com/patch_0220.txt

- Q11. Phison aiDAPTIV+ not support!

- Confirm the disk status and check if the AI100 is mounted.

- ◆ lsblk
- ◆ df -h

- phisonai2 command

- ◆ Provide the phisonai2 command to confirm if nvme_path is running on AI100.

- The example from the customer
 - ◆ The output of lsblk shows a line with mdxxx, indicating a RAID array, typically associated with AI100. You can see that it is mounted to the Ubuntu system at /media/test/820f95...
 - ◆ phisonai2 --nvme_path 沒有跑在系統掛載的位置(/media/test/820f95...)
- Q12. How to observe Phison SSD Usage
 - Use the nmon tool to monitor.
- Q13. Error Code: Attribute busid of node nic not found
 - use --specify_gpus to specify GPUs. If you want to use No3 and No2 GPU, set specify_gpus="3,2".
- Q14. Unable to JIT load the async_io op due to it not being compatible due to hardware/software issue.
 - sudo apt install libaio-dev
- Q15. Cmath error
 - sudo apt install libstdc++12-dev
- Q16. mdadm: cannot open /dev/nvme0n1p1: Device or resource busy
 - It may be due to unresolved RAID. Run sudo mdadm stop


```
sudo mdadm --stop /dev/md*
```
- Q17. Permission denied: '/media/user/md4'
 - sudo chmod 777 -R /media/user/md4
(<https://github.com/microsoft/DeepSpeed/pull/4538>)
- Q18. You need to have sentencepiece installed to convert a slow tokenizer to a fast one.
 - It may be due to unresolved RAID. Run sudo mdadm stop


```
pip install --user sentencepiece
```
- Q19. vicuna 33b about protobuf
 - It may be due to unresolved RAID. Run sudo mdadm stop


```
pip install --upgrade protobuf
```
- Q20. GPU Fan Error
 - <https://askubuntu.com/questions/16371/how-do-i-disable-x-at-boot-time-so-that-the-system-boots-in-text-mode>

- Q21. /dev/ubuntu-vg/ubuntu-lv out of space

-

```
sudo lvextend -l +100%FREE /dev/ubuntu-vg/ubuntu-lv
sudo resize2fs /dev/ubuntu-vg/ubuntu-lv
```

- Q22. ImportError: libcudnn.so.8: cannot open shared object file: No such file or directory

- uninstall the existing version of torch and reinstall torch with CUDA support.

```
pip install torch==2.1.2 torchvision==0.16.2 torchaudio==2.1.2 --index-url \
https://download.pytorch.org/whl/cu121
```

- Q23. ImportError: libcudnn.so.11: cannot open shared object file: No such file or directory

- Reinstall transformers and flash-attn.

```
pip uninstall transformers flash-attn
```

```
pip install transformers==4.35.2 flash_attn-2.4.2+cu122torch2.1cxx11abiFALSE-cp310-cp310-
linux_x86_64.whlQ24. vicuna 33b about protobuf
```

- Q24. RuntimeError: Ninja is required to load C++ extensions

- If in a virtual environment, ensure to use the outermost ninja module by running conda deactivate to the outer layer, then:

```
conda deactivate
pip install ninja
```

- Q25. RuntimeError: Ninja is requ

```
conda deactivate
libcusparse.so.11: cannot open shared object file: No such file or directory
CUDA SETUP: Something unexpected happened. Please compile from source:
git clone https://github.com/TimDettmers/bitsandbytes.git
cd bitsandbytes
```

- First run `pip list | grep nv` to confirm if the CUDA version matches the version installed with torch. If they are different, uninstall and reinstall torch
- <https://github.com/TimDettmers/bitsandbytes/issues/63> : `sudo ldconfig /usr/local/cuda/lib64`
- There may be an issue in the root environment.

- Q26. Encountering ninja build issue, missing python.h

- Refer to <https://stackoverflow.com/questions/21530577/fatal-error-python-h-no-such-file-or-directory>

- Q27. How to install aiDAPTIV if I use RockyLinux 9.3
 - Please update your Linux kernel version to 6.8.9 first, and follow the installation steps above.
- Q28. How to install aiDAPTIV if I use RockyLinux 9.3

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
torch 2.7.0+cu128 requires nvidia-nccl-cu12==2.26.2; platform_system == "linux" and platform_machine == "x86_64", but you have nvidia-nccl-cu12 2.27.7 which is incompatible.

- Please don't worry if the above error occurs, it will not affect subsequent training.
- Q29. If you encounter the AttributeError: 'BaichuanModel' object has no attribute 'param_name' error while running validation with Baichuan, you can simply comment out the red section in get_alibi_mask() (the else block in the picture):

```

314         self.embed_tokens = value
315
316     def get_alibi_mask(self, tensor, seq_length_with_past):
317         if self.training:
318             slopes = torch.Tensor(_get_interleave(self.n_head))
319             position_point = (
320                 torch.arange(seq_length_with_past) - seq_length_with_past + 1
321             )
322             position_point = (
323                 position_point.unsqueeze(0)
324                 .unsqueeze(0)
325                 .expand(self.n_head, seq_length_with_past, -1)
326             )
327             diag = torch.diag(position_point[0])
328             position_point = position_point - diag.unsqueeze(0).unsqueeze(0).transpose(
329                 -1, -2
330             )
331             alibi = slopes.unsqueeze(1).unsqueeze(1) * position_point
332             mask = _buffered_future_mask(
333                 tensor, seq_length_with_past, alibi, self.n_head
334             )
335         else:
336             if self.first_run:
337                 self.first_run = False
338                 self.register_buffer(
339                     "future_mask",
340                     _gen_alibi_mask(tensor, self.n_head, self.max_cache_pos).to(
341                         tensor
342                     ),
343                     persistent=False,
344                 )
345             if seq_length_with_past > self.max_cache_pos:
346                 self.max_cache_pos = seq_length_with_past
347                 self.register_buffer(
348                     "future_mask",
349                     _gen_alibi_mask(tensor, self.n_head, self.max_cache_pos).to(
350                         tensor
351                     ),
352                     persistent=False,
353                 )
354             mask = self.future_mask[
355                 : self.n_head, :seq_length_with_past, :seq_length_with_past
356             ]
357         return mask

```

- Q30. Why does the training loss remain consistently at 0.0 when using Triton, or why does it always show as NAN without Triton when working with Mistral-7B-v0.3?

- This situation arises because the model learns effectively from a simple data distribution, which is due to the lack of conditional guidance. Since Mistral-7B-v0.3 does not utilize a system prompt within the chat_template, this can lead to issues such as consistently zero or NAN training losses. To improve data randomness, we recommend modifying the chat_template field in the tokenizer_config.json file to incorporate a system prompt into the input. Please replace the “loop.last” as follows with “loop.first”.

○

```
173
174
175
176 {%- endif %}\n          {%- if loop.last and system_message is defined %}\n          {{- \"[INST] \" + system_message + \"\\n\\n\" + message[\"content\"] + \"[/INST]\" }}\n177
178
179
```